Financial Time-Series Forecasting with LSTM and GRU Networks

Academic Deep Learning Project | Starter Notebook

This notebook builds a reproducible forecasting experiment on daily NIFTY 50 index data. It predicts the **next trading day's log return** from a rolling window of past market features, then converts the predicted return into an estimated next closing level for visualization.

What you will build

- Download and inspect historical market data.
- Engineer features using only information available at each forecast origin.
- Split examples chronologically into train, validation and test sets.
- Fit preprocessing scalers on the training period only.
- Compare a zero-return persistence baseline with SimpleRNN, LSTM and GRU models.
- Evaluate return errors, closing-level errors and directional accuracy.
- Visualize test-period predictions and training curves.

> **Important:** This is an educational forecasting experiment, not investment advice. A good forecast metric does not establish that a trading strategy is profitable. Historical data can be revised or incomplete, and market behavior changes. Do not use this notebook to make real-money trading decisions.

Data source: Yahoo Finance historical data for NIFTY 50 (`^NSEI`), accessed with `yfinance`. This is a public data endpoint and may change; rerun the download cell to refresh the dataset.

1. Install dependencies

Run this cell in Google Colab or another Python environment with internet access.

```
%pip -q install yfinance tensorflow scikit-learn matplotlib pandas numpy
```

2. Import packages and set the experiment configuration

```
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf
import tensorflow as tf

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
from IPython.display import display

SEED = 42
np.random.seed(SEED)
tf.keras.utils.set_random_seed(SEED)

TICKER = "^NSEI"       # NIFTY 50 index on Yahoo Finance
PERIOD = "max"         # Download available daily history
LOOKBACK = 30          # Past trading sessions used for each prediction
TRAIN_FRACTION = 0.70
VALIDATION_FRACTION = 0.15
EPOCHS = 40
BATCH_SIZE = 32

print("TensorFlow version:", tf.__version__)
print("Ticker:", TICKER, "| Lookback:", LOOKBACK, "trading sessions")
```

```
TensorFlow version: 2.21.0
Ticker: ^NSEI | Lookback: 30 trading sessions
```

3. Download and inspect historical data

We use OHLC price columns only, so this project does not depend on the sometimes-missing volume field for an index. The model predicts a one-day-ahead return from each day's available information.

```
raw = yf.Ticker(TICKER).history(period=PERIOD, interval="1d", auto_adjust

if raw is None or raw.empty:
    raise RuntimeError(
        "No data was returned. Check your internet connection, ticker sym
        "or Yahoo Finance availability, then retry this cell."
    )

required_columns = ["Open", "High", "Low", "Close"]
missing = [col for col in required_columns if col not in raw.columns]
if missing:
    raise ValueError(f"Downloaded data is missing required columns: {miss

data = raw[required_columns].copy()
```

```
data = data.apply(pd.to_numeric, errors="coerce")
data = data.dropna(subset=required_columns)
data = data[~data.index.duplicated(keep="last")].sort_index()

if len(data) < 500:
    raise ValueError(f"Only {len(data)} rows downloaded; more history is

print(f"Rows: {len(data):,}")
print("Date range:", data.index.min().date(), "to", data.index.max().date
display(data.head())
display(data.tail())
```

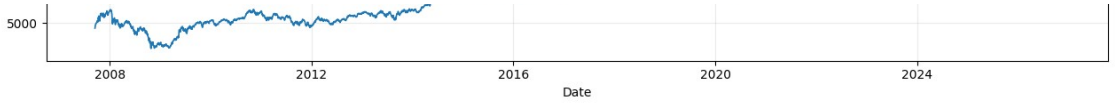
```
Rows: 4,690
Date range: 2007-09-17 to 2026-10-29
```

Date	Open	High	Low	Close
2007-09-17 00:00:00+05:30	4518.450195	4549.049805	4482.850098	4494.649902
2007-09-18 00:00:00+05:30	4494.100098	4551.799805	4481.549805	4546.200195
2007-09-19 00:00:00+05:30	4550.250000	4739.000000	4550.250000	4732.350098
2007-09-20 00:00:00+05:30	4734.850098	4760.850098	4721.149902	4747.549805
2007-09-21 00:00:00+05:30	4752.950195	4855.700195	4733.700195	4837.549805

Date	Open	High	Low	Close
2026-10-23 00:00:00+05:30	22742.199219	22787.250000	22372.000000	22600.000000
2026-10-26 00:00:00+05:30	22524.400391	22753.199219	22387.349609	22753.199219

```
fig, ax = plt.subplots(figsize=(13, 4))
ax.plot(data.index, data["Close"], linewidth=1.2)
ax.set_title(f"{TICKER} — Daily Closing Level")
ax.set_xlabel("Date")
ax.set_ylabel("Index level")
ax.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```





4. Feature engineering and target alignment

Forecast definition: At the end of trading day *t*, use features through day *t* to forecast the log return from close *t* to close *t+1*. The target is therefore `log(Close[t+1] / Close[t])`. Every input feature is built from information available on or before day *t*.

```
close = data["Close"]
log_return = np.log(close).diff()

features = pd.DataFrame(index=data.index)
features["log_return_1d"] = log_return
features["intraday_range"] = (data["High"] - data["Low"]) / close
features["open_close_return"] = (close - data["Open"]) / data["Open"]
features["momentum_5d"] = close.pct_change(5)
features["momentum_10d"] = close.pct_change(10)
features["volatility_5d"] = log_return.rolling(5).std()
features["volatility_20d"] = log_return.rolling(20).std()
features["sma_gap_10d"] = close / close.rolling(10).mean() - 1.0
features["sma_gap_20d"] = close / close.rolling(20).mean() - 1.0
features["sma_gap_50d"] = close / close.rolling(50).mean() - 1.0

FEATURE_COLUMNS = list(features.columns)
next_day_log_return = log_return.shift(-1)
next_day_date = pd.Series(data.index, index=data.index).shift(-1)

model_df = features.copy()
model_df["target_return"] = next_day_log_return
model_df["target_date"] = next_day_date
model_df["close_price"] = close
model_df = model_df.replace([np.inf, -np.inf], np.nan)
model_df = model_df.dropna(subset=FEATURE_COLUMNS + ["target_return", "ta

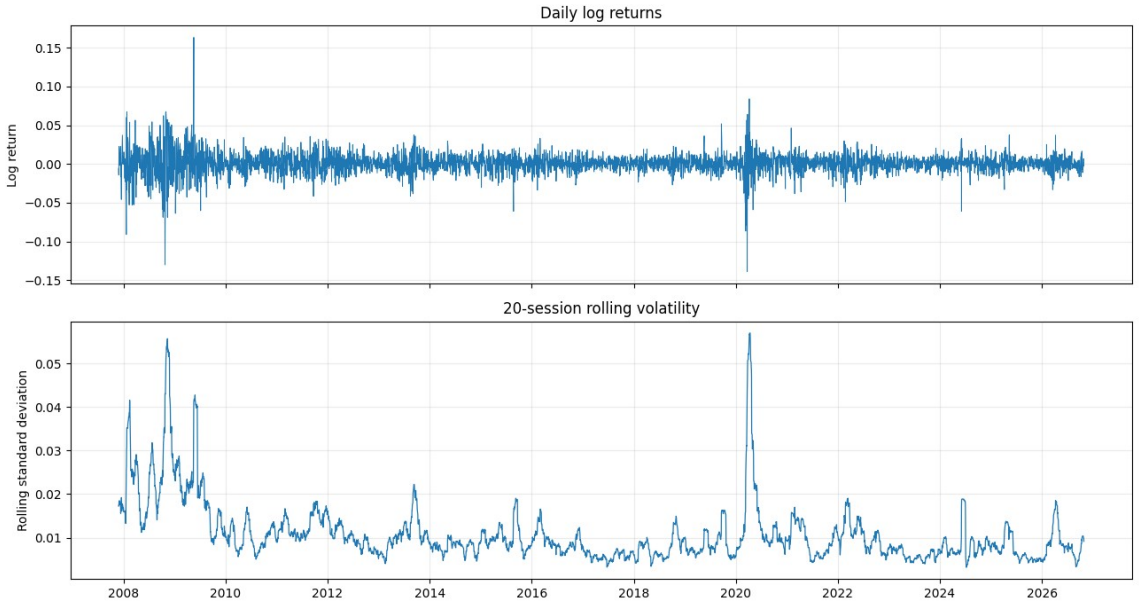
print("Features:", FEATURE_COLUMNS)
print("Usable feature rows:", f"{len(model_df):,}")
display(model_df[FEATURE_COLUMNS + ["target_return", "target_date"]].head
```

```
Features: ['log_return_1d', 'intraday_range', 'open_close_return', 'moment
```

```
Usable feature rows: 4,640
```

Date	log_return_1d	intraday_range	open_close_return	momentum_
2007-11-27 00:00:00+05:30	-0.005871	0.015435	-0.005428	-0.0143
2007-11-28 00:00:00+05:30	-0.014246	0.027494	-0.014387	0.0101
2007-11-29 00:00:00+05:30	0.003031	0.020037	0.002991	0.0208
2007-11-30 00:00:00+05:30	0.022489	0.026012	0.022870	0.0274
2007-12-03 00:00:00+05:30	0.017588	0.021176	0.017267	0.0232

```
fig, axes = plt.subplots(2, 1, figsize=(13, 7), sharex=True)
axes[0].plot(model_df.index, model_df["log_return_1d"], linewidth=0.7)
axes[0].set_title("Daily log returns")
axes[0].set_ylabel("Log return")
axes[0].grid(alpha=0.25)
axes[1].plot(model_df.index, model_df["volatility_20d"], linewidth=0.9)
axes[1].set_title("20-session rolling volatility")
axes[1].set_ylabel("Rolling standard deviation")
axes[1].grid(alpha=0.25)
plt.tight_layout()
plt.show()
```



5. Build rolling sequences and create chronological splits

We do **not** randomly shuffle this time series. The earliest 70% of forecast examples are used for training, the next 15% for validation, and the final 15% for the held-out test. Overlapping lookback windows are expected in time-series forecasting; no future targets are used to train the models.

```
feature_matrix = model_df[FEATURE_COLUMNS].to_numpy(dtype=np.float32)
target_all_rows = model_df["target_return"].to_numpy(dtype=np.float32)
end_positions = np.arange(LOOKBACK - 1, len(model_df))

if len(end_positions) < 300:
    raise ValueError("Not enough sequences after applying the lookback wi

# Each sequence ends on feature date t and its target is the return to th
sample_end_dates = model_df.index[end_positions]
target_dates = model_df["target_date"].iloc[end_positions].to_numpy()
y_all = target_all_rows[end_positions]
previous_close_all = model_df["close_price"].iloc[end_positions].to_numpy

n_samples = len(end_positions)
train_end = int(n_samples * TRAIN_FRACTION)
val_end = int(n_samples * (TRAIN_FRACTION + VALIDATION_FRACTION))

if train_end <= 0 or val_end <= train_end or val_end >= n_samples:
    raise ValueError("Invalid split sizes. Adjust TRAIN_FRACTION or VALID

train_idx = np.arange(0, train_end)
val_idx = np.arange(train_end, val_end)
test_idx = np.arange(val_end, n_samples)

# Fit the feature scaler only on feature rows available by the end of the
feature_fit_end_date = sample_end_dates[train_end - 1]
x_scaler = StandardScaler()
x_scaler.fit(model_df.loc[:feature_fit_end_date, FEATURE_COLUMNS].to_nump
scaled_features = x_scaler.transform(feature_matrix).astype(np.float32)

X_all = np.stack([
    scaled_features[end - LOOKBACK + 1 : end + 1]
    for end in end_positions
]).astype(np.float32)
```

```
# Fit the target scaler only on training targets.
y_scaler = StandardScaler()
y_scaler.fit(y_all[train_idx].reshape(-1, 1))
y_scaled = y_scaler.transform(y_all.reshape(-1, 1)).astype(np.float32)

X_train, y_train = X_all[train_idx], y_scaled[train_idx]
X_val, y_val = X_all[val_idx], y_scaled[val_idx]
X_test = X_all[test_idx]
y_test = y_all[test_idx]  # Keep the unscaled values for evaluation.

print(f"Samples: total={n_samples:,} | train={len(train_idx):,} | validat
print("Training target period ends:", pd.to_datetime(target_dates[train_e
print("Validation target period:", pd.to_datetime(target_dates[train_end]
print("Test target period:", pd.to_datetime(target_dates[val_end]).date()
print("X_train shape:", X_train.shape, "| X_val shape:", X_val.shape, "|
print("Input shape = (samples, lookback steps, features)")
```

```
Samples: total=4,611 | train=3,227 | validation=692 | test=692
Training target period ends: 2021-03-22
Validation target period: 2021-03-23 to 2024-01-08
Test target period: 2024-01-09 to 2026-10-29
X_train shape: (3227, 30, 10) | X_val shape: (692, 30, 10) | X_test shape:
Input shape = (samples, lookback steps, features)
```

6. Define SimpleRNN, LSTM and GRU models

All neural models receive the same input sequences and use the same optimizer, loss function, validation period and early-stopping policy. This makes the comparison more interpretable, although it does not guarantee that each architecture is optimally tuned.

```
def build_recurrent_model(model_type: str, input_shape: tuple) -> tf.kera
    """Build a small recurrent regression model."""
    model_type = model_type.upper()
    model = tf.keras.Sequential(name=f"{model_type.lower()}_forecaster")
    model.add(tf.keras.layers.Input(shape=input_shape))

    if model_type == "RNN":
        model.add(tf.keras.layers.SimpleRNN(32, return_sequences=True))
        model.add(tf.keras.layers.Dropout(0.2))
        model.add(tf.keras.layers.SimpleRNN(16))
    elif model_type == "LSTM":
        model.add(tf.keras.layers.LSTM(64, return_sequences=True))
        model.add(tf.keras.layers.Dropout(0.2))
        model.add(tf.keras.layers.LSTM(32))
    elif model_type == "GRU":
        model.add(tf.keras.layers.GRU(64, return_sequences=True))
        model.add(tf.keras.layers.Dropout(0.2))
        model.add(tf.keras.layers.GRU(32))
    else:
```

```
            raise ValueError("model_type must be one of: 'RNN', 'LSTM', 'GRU'

        model.add(tf.keras.layers.Dense(16, activation="relu"))
        model.add(tf.keras.layers.Dense(1, activation="linear"))
        model.compile(
            optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
            loss=tf.keras.losses.Huber(),
            metrics=[tf.keras.metrics.MeanAbsoluteError(name="mae")],
        )
        return model


    def make_callbacks():
        return [
            tf.keras.callbacks.EarlyStopping(
                monitor="val_loss", patience=8, restore_best_weights=True, ve
            ),
            tf.keras.callbacks.ReduceLROnPlateau(
                monitor="val_loss", factor=0.5, patience=4, min_lr=1e-5, verb
            ),
        ]

    input_shape = (X_train.shape[1], X_train.shape[2])
    print("Model input shape:", input_shape)
    print("Number of input features:", len(FEATURE_COLUMNS))
```

```
Model input shape: (30, 10)
Number of input features: 10
```

7. Train the models

This cell trains three models. On a CPU it can take a few minutes; Colab's GPU runtime may help, though these models are small. `shuffle=False` preserves the order of training examples.

```
    models = {}
    histories = {}

    for model_type in ["RNN", "LSTM", "GRU"]:
        print("\\n" + "=" * 18, f"Training {model_type}", "=" * 18)
        tf.keras.backend.clear_session()
        tf.keras.utils.set_random_seed(SEED)
        model = build_recurrent_model(model_type, input_shape)
        model.summary()
        history = model.fit(
            X_train, y_train,
            validation_data=(X_val, y_val),
            epochs=EPOCHS,
            batch_size=BATCH_SIZE,
            shuffle=False,
            callbacks=make_callbacks(),
```

```
        verbose=1,
    )
    models[model_type] = model
    histories[model_type] = history

print("Finished training:", list(models.keys()))
```

```
\n================== Training RNN ==================
Model: "rnn_forecaster"
```

Layer (type)	Output Shape	Param #
simple_rnn (SimpleRNN)	(None, 30, 32)	1,376
dropout (Dropout)	(None, 30, 32)	0
simple_rnn_1 (SimpleRNN)	(None, 16)	784
dense (Dense)	(None, 16)	272
dense_1 (Dense)	(None, 1)	17

```
 Total params: 2,449 (9.57 KB)
 Trainable params: 2,449 (9.57 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 14s 51ms/step - loss: 0.3663 - mae: 0.6972 -
Epoch 2/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 11ms/step - loss: 0.3476 - mae: 0.6697 - v
Epoch 3/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.3427 - mae: 0.6655 - va
Epoch 4/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.3420 - mae: 0.6628 - va
Epoch 5/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.3359 - mae: 0.6555 - va
Epoch 6/40
 93/101 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.5576 - mae: 0.9120
Epoch 6: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.3380 - mae: 0.6569 - va
Epoch 7/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.3315 - mae: 0.6492 - va
Epoch 8/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.3325 - mae: 0.6498 - va
Epoch 9/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.3299 - mae: 0.6474 - va
Epoch 10/40
 93/101 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 0.5444 - mae: 0.8969
Epoch 10: ReduceLROnPlateau reducing learning rate to 0.0002500000011874362
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 0.3299 - mae: 0.6477 - va
Epoch 10: early stopping
Restoring model weights from the end of the best epoch: 2.
\n================== Training LSTM ==================
Model: "lstm_forecaster"
```

Layer (type)	Output Shape	Param #
lstm (LSTM)	(None, 30, 64)	19,200
dropout (Dropout)	(None, 30, 64)	0
lstm_1 (LSTM)	(None, 32)	12,416

lstm_1 (LSTM)	(None, 32)	12,416
dense (Dense)	(None, 16)	528
dense_1 (Dense)	(None, 1)	17

Total params: 32,161 (125.63 KB)
Trainable params: 32,161 (125.63 KB)
Non-trainable params: 0 (0.00 B)

```
Epoch 1/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 6s 13ms/step - loss: 0.3408 - mae: 0.6612 - v
Epoch 2/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3398 - mae: 0.6588 - v
Epoch 3/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3394 - mae: 0.6586 - v
Epoch 4/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3394 - mae: 0.6582 - v
Epoch 5/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3392 - mae: 0.6581 - v
Epoch 6/40
 98/101 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.5490 - mae: 0.9012
Epoch 6: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3390 - mae: 0.6577 - v
Epoch 7/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3388 - mae: 0.6577 - v
Epoch 8/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3378 - mae: 0.6564 - v
Epoch 9/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3373 - mae: 0.6556 - v
Epoch 10/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3364 - mae: 0.6550 - v
Epoch 11/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3359 - mae: 0.6544 - v
Epoch 12/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3363 - mae: 0.6551 - v
Epoch 13/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3350 - mae: 0.6533 - v
Epoch 14/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3337 - mae: 0.6519 - v
Epoch 15/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3327 - mae: 0.6509 - v
Epoch 16/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3338 - mae: 0.6522 - v
Epoch 17/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3316 - mae: 0.6497 - v
Epoch 18/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3297 - mae: 0.6479 - v
Epoch 19/40
 96/101 ━━━━━━━━━━━━━━━━━━━━ 0s 9ms/step - loss: 0.5304 - mae: 0.8847
Epoch 19: ReduceLROnPlateau reducing learning rate to 0.0002500000011874362
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3282 - mae: 0.6469 - v
Epoch 20/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - loss: 0.3252 - mae: 0.6429 - v
Epoch 21/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 13ms/step - loss: 0.3217 - mae: 0.6392 - v
Epoch 22/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 2s 10ms/step - loss: 0.3236 - mae: 0.6406 - v
Epoch 23/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.5086 - mae: 0.8603
```

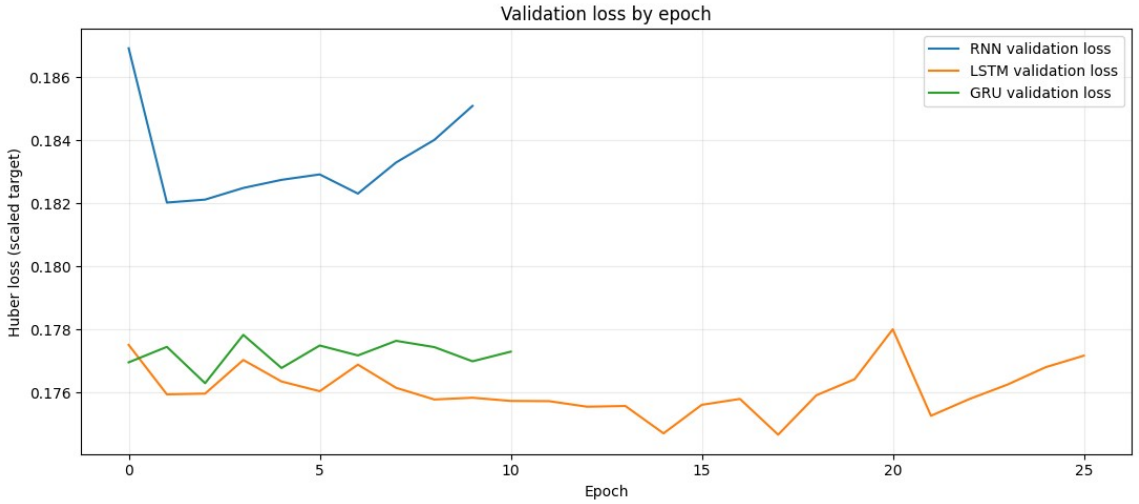
```
Epoch 23: ReduceLROnPlateau reducing learning rate to 0.000125000005937181
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3233 - mae: 0.6412 - v
Epoch 24/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3177 - mae: 0.6345 - v
Epoch 25/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3166 - mae: 0.6324 - v
Epoch 26/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3141 - mae: 0.6303 - v
Epoch 26: early stopping
Restoring model weights from the end of the best epoch: 18.
\n================== Training GRU ==================
Model: "gru_forecaster"
```

Layer (type)	Output Shape	Param #
gru (GRU)	(None, 30, 64)	14,592
dropout (Dropout)	(None, 30, 64)	0
gru_1 (GRU)	(None, 32)	9,408
dense (Dense)	(None, 16)	528
dense_1 (Dense)	(None, 1)	17

```
 Total params: 24,545 (95.88 KB)
 Trainable params: 24,545 (95.88 KB)
 Non-trainable params: 0 (0.00 B)
Epoch 1/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 3s 13ms/step - loss: 0.3437 - mae: 0.6649 - v
Epoch 2/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3400 - mae: 0.6599 - v
Epoch 3/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3390 - mae: 0.6590 - v
Epoch 4/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3379 - mae: 0.6576 - v
Epoch 5/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3367 - mae: 0.6567 - v
Epoch 6/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3356 - mae: 0.6557 - v
Epoch 7/40
 98/101 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.5385 - mae: 0.8925
Epoch 7: ReduceLROnPlateau reducing learning rate to 0.0005000000237487257
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3355 - mae: 0.6553 - v
Epoch 8/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3314 - mae: 0.6501 - v
Epoch 9/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3310 - mae: 0.6496 - v
Epoch 10/40
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3303 - mae: 0.6485 - v
Epoch 11/40
 95/101 ━━━━━━━━━━━━━━━━━━━━ 0s 8ms/step - loss: 0.5342 - mae: 0.8877
Epoch 11: ReduceLROnPlateau reducing learning rate to 0.00025000000118743622
101/101 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 0.3304 - mae: 0.6491 - v
Epoch 11: early stopping
Restoring model weights from the end of the best epoch: 3.
Finished training: ['RNN', 'LSTM', 'GRU']
```

```
fig, ax = plt.subplots(figsize=(11, 5))
```

```
for model_type, history in histories.items():
    ax.plot(history.history["val_loss"], label=f"{model_type} validation
ax.set_title("Validation loss by epoch")
ax.set_xlabel("Epoch")
ax.set_ylabel("Huber loss (scaled target)")
ax.grid(alpha=0.25)
ax.legend()
plt.tight_layout()
plt.show()
```



8. Evaluate on the untouched test period

The main regression target is a next-session log return. We also transform each predicted return into an estimated next close using the close on the forecast-origin date. The zero-return baseline predicts that tomorrow's close will equal today's close. It is a simple but important reference point.

```
def calculate_metrics(y_true_return, y_pred_return, previous_close, inclu
    """Metrics in return units and reconstructed close-level units."""
    y_true_return = np.asarray(y_true_return).reshape(-1)
    y_pred_return = np.asarray(y_pred_return).reshape(-1)
    previous_close = np.asarray(previous_close).reshape(-1)

    actual_close = previous_close * np.exp(y_true_return)
```

```
    actual_close = previous_close * np.exp(y_true_return)
    predicted_close = previous_close * np.exp(y_pred_return)

    row = {
        "Return MAE (pp)": mean_absolute_error(y_true_return, y_pred_retu
        "Return RMSE (pp)": np.sqrt(mean_squared_error(y_true_return, y_p
        "Close MAE (points)": mean_absolute_error(actual_close, predicted
        "Close RMSE (points)": np.sqrt(mean_squared_error(actual_close, p
    }
    if include_direction:
        true_direction = np.sign(y_true_return)
        pred_direction = np.sign(y_pred_return)
        row["Direction accuracy (%)"] = np.mean(true_direction == pred_di
    else:
        row["Direction accuracy (%)"] = np.nan
    return row

# Inverse-transform test predictions to log-return units.
predictions = {}
for model_type, model in models.items():
    pred_scaled = model.predict(X_test, verbose=0)
    predictions[model_type] = y_scaler.inverse_transform(pred_scaled).res

# Persistence baseline: forecast no change in closing level (zero log ret
predictions["Zero-return baseline"] = np.zeros_like(y_test)

previous_close_test = previous_close_all[test_idx]
metric_rows = {}
for model_type, pred_return in predictions.items():
    metric_rows[model_type] = calculate_metrics(
        y_test,
        pred_return,
        previous_close_test,
        include_direction=(model_type != "Zero-return baseline"),
    )

metrics_df = pd.DataFrame.from_dict(metric_rows, orient="index")
metrics_df.index.name = "Model"
metrics_df = metrics_df.sort_values("Return RMSE (pp)")
display(metrics_df.round(4))

print("Metric notes:")
print("• Return error metrics are expressed in percentage points (pp), no
print("• Close-level errors are measured in NIFTY 50 index points.")
print("• Direction accuracy is omitted for the zero-return baseline becau
```

Model	Return MAE (pp)	Return RMSE (pp)	Close MAE (points)	Close RMSE (points)	Direction accuracy (%)
GRU	0.6087	0.8529	145.3246	201.7478	52.6012
Zero-return	0.6117	0.8540	146.0002	201.9750	NaN

Zero-return baseline	0.6117	0.8340	146.0662	201.9790	NaN
LSTM	0.6156	0.8563	147.0127	202.6003	50.1445
RNN	0.6271	0.8750	149.7043	206.9242	48.5549

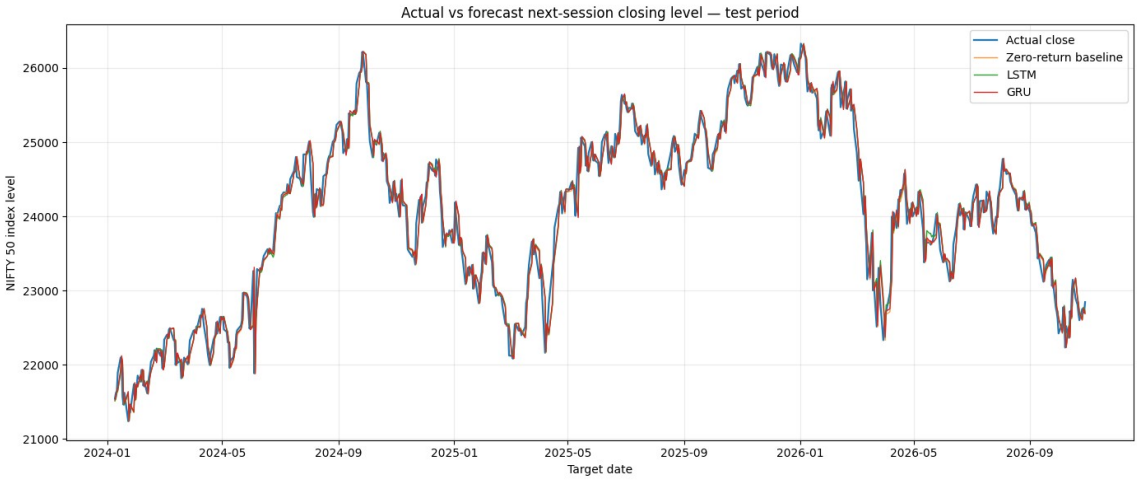
Metric notes:

9. Visualize predictions on the test set

```
test_target_dates = pd.to_datetime(target_dates[test_idx])
actual_close_test = previous_close_test * np.exp(y_test)
forecast_df = pd.DataFrame({
    "Target date": test_target_dates,
    "Previous close": previous_close_test,
    "Actual next close": actual_close_test,
    "Zero-return baseline": previous_close_test,
})
for model_type in ["RNN", "LSTM", "GRU"]:
    forecast_df[f"{model_type} predicted close"] = previous_close_test *

fig, ax = plt.subplots(figsize=(14, 6))
ax.plot(forecast_df["Target date"], forecast_df["Actual next close"], lab
ax.plot(forecast_df["Target date"], forecast_df["Zero-return baseline"],
ax.plot(forecast_df["Target date"], forecast_df["LSTM predicted close"],
ax.plot(forecast_df["Target date"], forecast_df["GRU predicted close"], l
ax.set_title("Actual vs forecast next-session closing level — test period
ax.set_xlabel("Target date")
ax.set_ylabel("NIFTY 50 index level")
ax.grid(alpha=0.25)
ax.legend()
plt.tight_layout()
plt.show()

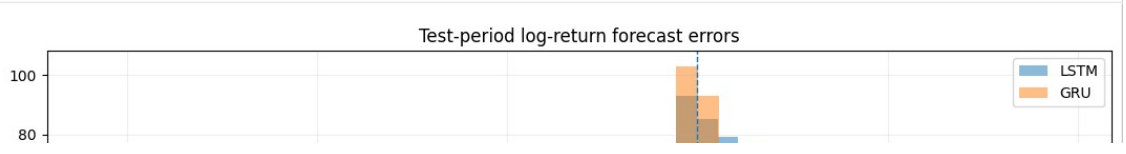
display(forecast_df.tail(10).round(3))
```

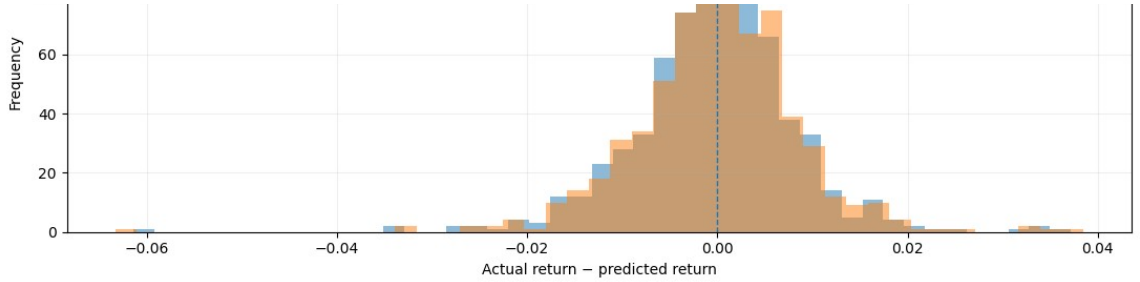


	Target date	Previous close	Actual next close	Zero-return baseline	RNN predicted close	LSTM predicted close	p
682	2026-10-15	22622.699	22997.349	22622.699	22579.824	22635.671	2

682	00:00:00+05:30	22622.699	22997.349	22622.699	22579.624	22655.071	2
683	2026-10-16 00:00:00+05:30	22997.350	23146.150	22997.350	22986.965	23011.896	2
684	2026-10-19 00:00:00+05:30	23146.150	22888.951	23146.150	23196.134	23165.893	2
685	2026-10-21 00:00:00+05:30	22888.949	22811.351	22888.949	22857.823	22911.853	2
686	2026-10-22 00:00:00+05:30	22811.350	22687.650	22811.350	22805.457	22835.209	2
687	2026-10-23 00:00:00+05:30	22687.650	22600.002	22687.650	22677.458	22709.449	2
688	2026-10-26 00:00:00+05:30	22600.000	22753.199	22600.000	22586.478	22619.772	2
689	2026-10-27 00:00:00+05:30	22753.199	22749.749	22753.199	22759.402	22773.013	2
690	2026-10-28 00:00:00+05:30	22749.750	22698.849	22749.750	22769.016	22767.695	2
691	2026-10-29 00:00:00+05:30	22698.850	22844.401	22698.850	22715.193	22714.809	2

```
# Return forecast errors: helpful for seeing whether predictions are bias
fig, ax = plt.subplots(figsize=(11, 4))
for model_type in ["LSTM", "GRU"]:
    errors = y_test - predictions[model_type]
    ax.hist(errors, bins=45, alpha=0.5, label=model_type)
ax.axvline(0, linestyle="--", linewidth=1)
ax.set_title("Test-period log-return forecast errors")
ax.set_xlabel("Actual return − predicted return")
ax.set_ylabel("Frequency")
ax.legend()
ax.grid(alpha=0.2)
plt.tight_layout()
plt.show()
```





10. Save the experiment outputs

These files can be added to your project repository after you review the results. Avoid committing very large raw datasets unless your assignment specifically requires them. Model files can be large, so consider Git LFS or keeping them in release artifacts.

```
from pathlib import Path

OUTPUT_DIR = Path("outputs")
OUTPUT_DIR.mkdir(exist_ok=True)

metrics_df.to_csv(OUTPUT_DIR / "model_metrics.csv")
forecast_df.to_csv(OUTPUT_DIR / "test_forecasts.csv", index=False)

for model_type, model in models.items():
    model.save(OUTPUT_DIR / f"{model_type.lower()}_forecaster.keras")

print("Saved files:")
for path in sorted(OUTPUT_DIR.iterdir()):
    print("-", path, f"({path.stat().st_size / 1024:.1f} KB)")
```

```
Saved files:
- outputs/gru_forecaster.keras (326.7 KB)
- outputs/lstm_forecaster.keras (416.0 KB)
- outputs/model_metrics.csv (0.5 KB)
- outputs/rnn_forecaster.keras (67.6 KB)
- outputs/test_forecasts.csv (87.2 KB)
```

```
%pip -q install yfinance tensorflow scikit-learn matplotlib pandas numpy
```

```
import warnings
```

```
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)

import json
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf
import tensorflow as tf

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
from IPython.display import display

SEED = 42
np.random.seed(SEED)
tf.keras.utils.set_random_seed(SEED)

TICKER = "^NSEI"       # NIFTY 50 index on Yahoo Finance
PERIOD = "max"         # Download available daily history
LOOKBACK = 30          # Baseline window used to derive chronological dat
TRAIN_FRACTION = 0.70
VALIDATION_FRACTION = 0.15
BATCH_SIZE = 32

print("TensorFlow version:", tf.__version__)
print("Ticker:", TICKER, "| Base lookback:", LOOKBACK, "trading sessions"
```

```
TensorFlow version: 2.21.0
Ticker: ^NSEI | Base lookback: 30 trading sessions
```

Experiment 2 — Lookback Sensitivity and Walk-Forward Validation

Project: Financial Time-Series Forecasting with Recurrent Neural Networks
Dataset: NIFTY 50 daily OHLC data from Yahoo Finance (`^NSEI`)
Main methods: GRU, expanding-window walk-forward evaluation, simple baselines

Research questions

1. Does a 20-, 30-, or 60-session lookback perform best on the development validation period?
2. Does a GRU outperform simple forecasting baselines across multiple sequential test folds?
3. Is performance stable across different market periods?

Important: This is an educational research project, not financial advice. The 2024–2026 test periods were examined in a prior run, so results from this notebook should be described as exploratory rather than as a pristine untouched final test. Do not use them repeatedly to select later features or models.

Run each cell from top to bottom. All core code is visible in notebook cells; no external `.py` file is required.

1. Install dependencies

```
%pip -q install yfinance tensorflow scikit-learn matplotlib pandas numpy
```

2. Import libraries and configure the dataset

```
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)

import json
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf
import tensorflow as tf

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
from IPython.display import display

SEED = 42
np.random.seed(SEED)
tf.keras.utils.set_random_seed(SEED)

TICKER = "^NSEI"       # NIFTY 50 index on Yahoo Finance
PERIOD = "max"         # Download available daily history
LOOKBACK = 30          # Baseline window used to derive chronological dat
TRAIN_FRACTION = 0.70
VALIDATION_FRACTION = 0.15
BATCH_SIZE = 32

print("TensorFlow version:", tf.__version__)
print("Ticker:", TICKER, "| Base lookback:", LOOKBACK, "trading sessions"
```

```
TensorFlow version: 2.21.0
Ticker: ^NSEI | Base lookback: 30 trading sessions
```

3. Download and inspect historical data

Yahoo Finance data availability can change. Record the date of your download in the final report.

```
raw = yf.Ticker(TICKER).history(
    period=PERIOD,
    interval="1d",
    auto_adjust=False,
    actions=False,
)

if raw is None or raw.empty:
    raise RuntimeError(
        "No data was returned. Check your internet connection, ticker sym
        "or Yahoo Finance availability, then retry this cell."
    )

required_columns = ["Open", "High", "Low", "Close"]
missing = [col for col in required_columns if col not in raw.columns]
if missing:
    raise ValueError(f"Downloaded data is missing required columns: {miss

data = raw[required_columns].copy()
data = data.apply(pd.to_numeric, errors="coerce")
data = data.dropna(subset=required_columns)
data = data[~data.index.duplicated(keep="last")].sort_index()

if len(data) < 500:
    raise ValueError(
        f"Only {len(data)} rows downloaded; more history is needed for a
    )

print(f"Rows downloaded: {len(data):,}")
print("Date range:", data.index.min().date(), "to", data.index.max().date
display(data.head())
display(data.tail())

fig, ax = plt.subplots(figsize=(13, 4))
ax.plot(data.index, data["Close"], linewidth=1.2)
ax.set_title(f"{TICKER} — Daily Closing Level")
ax.set_xlabel("Date")
ax.set_ylabel("Index level")
ax.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```

```
Rows downloaded: 4,690
Date range: 2007-09-17 to 2026-10-29
```

Date	Open	High	Low	Close

2007-09-17 00:00:00+05:30	4518.450195	4549.049805	4482.850098	4494.649902
2007-09-18 00:00:00+05:30	4494.100098	4551.799805	4481.549805	4546.200195
2007-09-19 00:00:00+05:30	4550.250000	4739.000000	4550.250000	4732.350098
2007-09-20 00:00:00+05:30	4734.850098	4760.850098	4721.149902	4747.549805
2007-09-21 00:00:00+05:30	4752.950195	4855.700195	4733.700195	4837.549805

Date	Open	High	Low	Close
2026-10-23 00:00:00+05:30	22742.199219	22787.250000	22372.000000	22600.000000
2026-10-26 00:00:00+05:30	22524.400391	22753.199219	22387.349609	22753.199219
2026-10-27 00:00:00+05:30	22708.800781	22829.750000	22636.849609	22749.750000
2026-10-28 00:00:00+05:30	22678.000000	22778.050781	22497.699219	22698.849609
2026-10-29 00:00:00+05:30	22690.650391	22874.150391	22656.750000	22852.949219



4. Engineer causal features and align the next-session target

At the end of day *t*, the features use data available through day *t*, and the target is the log return from close *t* to close *t+1*. The train/validation/test splits below remain chronological.

```
# Calculate features using information available on or before each foreca
close = data["Close"]
log_return = np.log(close).diff()

```

```
features = pd.DataFrame(index=data.index)
features["log_return_1d"] = log_return
features["intraday_range"] = (data["High"] - data["Low"]) / close
features["open_close_return"] = (close - data["Open"]) / data["Open"]
features["momentum_5d"] = close.pct_change(5)
features["momentum_10d"] = close.pct_change(10)
features["volatility_5d"] = log_return.rolling(5).std()
features["volatility_20d"] = log_return.rolling(20).std()
features["sma_gap_10d"] = close / close.rolling(10).mean() - 1.0
features["sma_gap_20d"] = close / close.rolling(20).mean() - 1.0
features["sma_gap_50d"] = close / close.rolling(50).mean() - 1.0

FEATURE_COLUMNS = list(features.columns)
next_day_log_return = log_return.shift(-1)
next_day_date = pd.Series(data.index, index=data.index).shift(-1)

model_df = features.copy()
model_df["target_return"] = next_day_log_return
model_df["target_date"] = next_day_date
model_df["close_price"] = close
model_df = model_df.replace([np.inf, -np.inf], np.nan)
model_df = model_df.dropna(
    subset=FEATURE_COLUMNS + ["target_return", "target_date", "close_pric
)

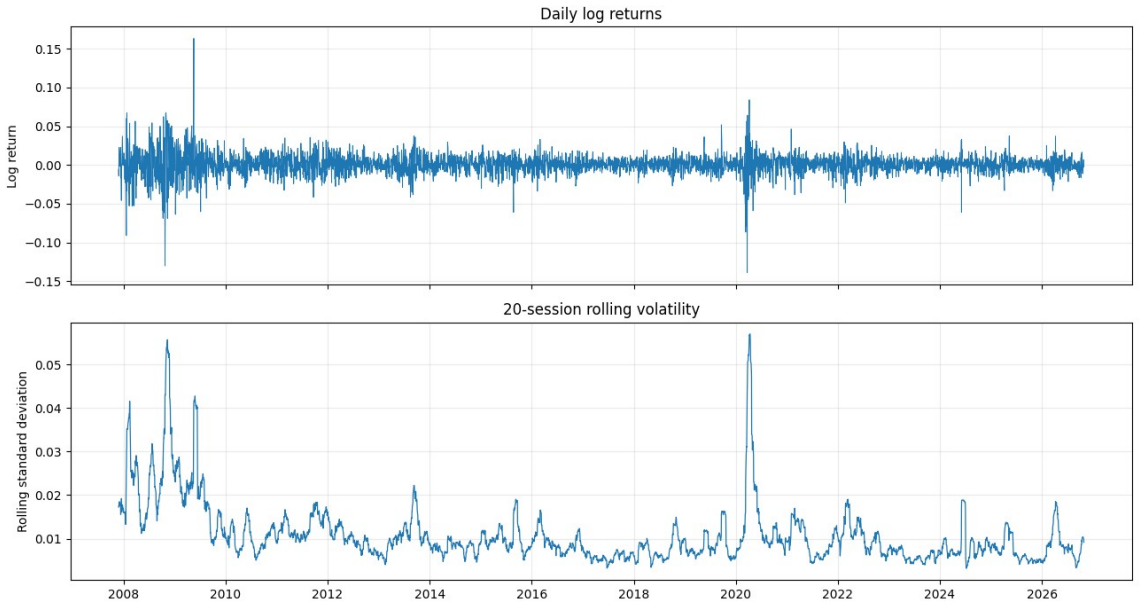
print("Features:", FEATURE_COLUMNS)
print("Usable feature rows:", f"{len(model_df):,}")
display(model_df[FEATURE_COLUMNS + ["target_return", "target_date"]].head

fig, axes = plt.subplots(2, 1, figsize=(13, 7), sharex=True)
axes[0].plot(model_df.index, model_df["log_return_1d"], linewidth=0.7)
axes[0].set_title("Daily log returns")
axes[0].set_ylabel("Log return")
axes[0].grid(alpha=0.25)
axes[1].plot(model_df.index, model_df["volatility_20d"], linewidth=0.9)
axes[1].set_title("20-session rolling volatility")
axes[1].set_ylabel("Rolling standard deviation")
axes[1].grid(alpha=0.25)
plt.tight_layout()
plt.show()
```

```
Features: ['log_return_1d', 'intraday_range', 'open_close_return', 'moment
Usable feature rows: 4,640
```

Date	log_return_1d	intraday_range	open_close_return	momentum_
2007-11-27 00:00:00+05:30	-0.005871	0.015435	-0.005428	-0.0143
2007-11-28 00:00:00+05:30	-0.014246	0.027494	-0.014387	0.0101
2007-11-29 00:00:00+05:30	0.003031	0.020037	0.002991	0.0208

2007-11-30 00:00:00+05:30	0.022489	0.026012	0.022870	0.0274
2007-12-03 00:00:00+05:30	0.017588	0.021176	0.017267	0.0232



5. Set up Experiment 2

The first code cell checks the prepared dataframe and reconstructs the same chronological date boundaries as the starter notebook. The lookback window is chosen using validation RMSE only.

```
from pathlib import Path
import json
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
import tensorflow as tf
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
from IPython.display import display

warnings.filterwarnings("ignore", category=FutureWarning)

# ------------------------ 0. Validate notebook context ----------------
required = ["model_df", "FEATURE_COLUMNS"]
missing = [name for name in required if name not in globals()]
if missing:
    raise RuntimeError(
        "Missing variables from the original notebook: " + ", ".join(miss
        + ". Run the original notebook through the feature-engineering ce
        "then run this file with `%run -i financial_timeseries_experiment
    )

if not isinstance(model_df, pd.DataFrame):
    raise TypeError("`model_df` must be the DataFrame created by the orig

needed_columns = set(FEATURE_COLUMNS) | {"target_return", "target_date",
missing_columns = sorted(needed_columns - set(model_df.columns))
if missing_columns:
    raise ValueError(f"model_df is missing required columns: {missing_col

SEED_EXP2 = int(globals().get("SEED", 42))
BATCH_EXP2 = int(globals().get("BATCH_SIZE", 32))
MAX_EPOCHS_EXP2 = 25
PATIENCE_EXP2 = 5
LOOKBACK_CANDIDATES = [20, 30, 60]
N_WALK_FORWARD_FOLDS = 3
TRAIN_FRACTION_EXP2 = float(globals().get("TRAIN_FRACTION", 0.70))
VALIDATION_FRACTION_EXP2 = float(globals().get("VALIDATION_FRACTION", 0.1

np.random.seed(SEED_EXP2)
tf.keras.utils.set_random_seed(SEED_EXP2)

feature_names = list(FEATURE_COLUMNS)
raw_features = model_df[feature_names].to_numpy(dtype=np.float32)
row_targets = model_df["target_return"].to_numpy(dtype=np.float32)
row_dates = pd.DatetimeIndex(pd.to_datetime(model_df["target_date"]))
row_previous_close = model_df["close_price"].to_numpy(dtype=np.float64)
row_last_return = model_df["log_return_1d"].to_numpy(dtype=np.float32)
n_rows = len(model_df)

# Recreate the original experiment's date boundaries using its original l
# if it exists. This keeps the same validation and test date ranges as Ex
base_lookback = int(globals().get("LOOKBACK", 30))
base_ends = np.arange(base_lookback - 1, n_rows)
base_dates = row_dates[base_ends]
base_n = len(base_ends)
base_train_end = int(base_n * TRAIN_FRACTION_EXP2)
base_val_end = int(base_n * (TRAIN_FRACTION_EXP2 + VALIDATION_FRACTION_EX
if base_train_end < 100 or base_val_end <= base_train_end or base_val_end
```

```
    raise ValueError("Not enough samples for the configured chronological

validation_start_date = base_dates[base_train_end]
test_start_date = base_dates[base_val_end]

print("=" * 76)
print("EXPERIMENT 2 — LOOKBACK SENSITIVITY + WALK-FORWARD VALIDATION")
print("=" * 76)
print(f"Rows available: {n_rows:,}")
print(f"Validation period starts: {validation_start_date}")
print(f"Held-out test period starts: {test_start_date}")
print(f"Lookback candidates: {LOOKBACK_CANDIDATES}")
print(f"Walk-forward folds: {N_WALK_FORWARD_FOLDS}")
print("The lookback sweep uses validation data only; test metrics are com
```

```
==========================================================================
EXPERIMENT 2 — LOOKBACK SENSITIVITY + WALK-FORWARD VALIDATION
==========================================================================
Rows available: 4,640
Validation period starts: 2021-03-23 00:00:00+05:30
Held-out test period starts: 2024-01-09 00:00:00+05:30
Lookback candidates: [20, 30, 60]
Walk-forward folds: 3
The lookback sweep uses validation data only; test metrics are computed la
```

6. Define sequence generation, GRU architecture, callbacks, and metrics

```
def _sample_arrays(lookback):
    """Create chronologically ordered rolling sequences and aligned targe
    if lookback < 1 or lookback >= n_rows:
        raise ValueError(f"Invalid lookback value: {lookback}")
    ends = np.arange(lookback - 1, n_rows)
    dates = row_dates[ends]
    y = row_targets[ends]
    previous_close = row_previous_close[ends]
    last_return = row_last_return[ends]
    return ends, dates, y, previous_close, last_return


def _sequences(scaled_features, sample_ends, lookback):
    return np.stack(
        [scaled_features[end - lookback + 1 : end + 1] for end in sample_
    ).astype(np.float32)


def _build_gru(input_shape):
    """Same two-layer GRU concept as Experiment 1, with compact callbacks
    model = tf.keras.Sequential(name="gru_forecaster_exp2")
    model.add(tf.keras.layers.Input(shape=input_shape))
    model.add(tf.keras.layers.GRU(64, return_sequences=True))
    model.add(tf.keras.layers.Dropout(0.2))
```

```
    model.add(tf.keras.layers.GRU(32))
    model.add(tf.keras.layers.Dense(16, activation="relu"))
    model.add(tf.keras.layers.Dense(1, activation="linear"))
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
        loss=tf.keras.losses.Huber(),
        metrics=[tf.keras.metrics.MeanAbsoluteError(name="mae")],
    )
    return model


def _callbacks():
    return [
        tf.keras.callbacks.EarlyStopping(
            monitor="val_loss", patience=PATIENCE_EXP2,
            restore_best_weights=True, verbose=0
        ),
        tf.keras.callbacks.ReduceLROnPlateau(
            monitor="val_loss", factor=0.5, patience=3,
            min_lr=1e-5, verbose=0
        ),
    ]


def _metrics(y_true, y_pred, previous_close, include_direction=True):
    """Return errors for log returns, reconstructed close levels, and opt
    y_true = np.asarray(y_true, dtype=np.float64).reshape(-1)
    y_pred = np.asarray(y_pred, dtype=np.float64).reshape(-1)
    previous_close = np.asarray(previous_close, dtype=np.float64).reshape
    actual_close = previous_close * np.exp(y_true)
    predicted_close = previous_close * np.exp(y_pred)
    return {
        "Return MAE (pp)": mean_absolute_error(y_true, y_pred) * 100,
        "Return RMSE (pp)": np.sqrt(mean_squared_error(y_true, y_pred)) *
        "Close MAE (points)": mean_absolute_error(actual_close, predicted
        "Close RMSE (points)": np.sqrt(mean_squared_error(actual_close, p
        "Direction accuracy (%)": (
            np.mean(np.sign(y_true) == np.sign(y_pred)) * 100
            if include_direction else np.nan
        ),
    }


def _fit_gru_for_fold(lookback, sample_ends, sample_dates, sample_y, trai
    """Fit a GRU using a fold-specific, training-only preprocessing pipel
    if len(train_ids) < 100 or len(val_ids) < 20:
        raise ValueError(
            f"Insufficient samples: train={len(train_ids)}, validation={l
        )

    # The feature scaler is fitted only through the final training foreca
    last_train_end = int(sample_ends[train_ids[-1]])
    x_scaler_fold = StandardScaler()
    x_scaler_fold.fit(raw_features[: last_train_end + 1])
```

```
    scaled_features_fold = x_scaler_fold.transform(raw_features).astype(n
    X_fold = _sequences(scaled_features_fold, sample_ends, lookback)

    y_scaler_fold = StandardScaler()
    y_scaler_fold.fit(sample_y[train_ids].reshape(-1, 1))
    y_scaled_fold = y_scaler_fold.transform(sample_y.reshape(-1, 1)).asty

    tf.keras.backend.clear_session()
    tf.keras.utils.set_random_seed(SEED_EXP2)
    model = _build_gru((lookback, len(feature_names)))
    history = model.fit(
        X_fold[train_ids], y_scaled_fold[train_ids],
        validation_data=(X_fold[val_ids], y_scaled_fold[val_ids]),
        epochs=MAX_EPOCHS_EXP2,
        batch_size=BATCH_EXP2,
        shuffle=False,
        callbacks=_callbacks(),
        verbose=0,
    )
    return model, history, X_fold, y_scaler_fold
```

7. Experiment 2A — Compare 20-, 30-, and 60-session lookback windows

The validation period selects the lookback. The test-fold metrics are not used for this selection.

```
# ------------------------- 1. Validation-only lookback sweep -----------
lookback_rows = []
lookback_validation_preds = {}
for lookback in LOOKBACK_CANDIDATES:
    sample_ends, sample_dates, sample_y, sample_prev_close, sample_last_r
    train_ids = np.flatnonzero(sample_dates < validation_start_date)
    val_ids = np.flatnonzero(
        (sample_dates >= validation_start_date) & (sample_dates < test_st
    )

    print(f"\n[Lookback sweep] Training GRU with lookback={lookback} ..."
    model, history, X_scaled, y_scaler = _fit_gru_for_fold(
        lookback, sample_ends, sample_dates, sample_y, train_ids, val_ids
    )
    val_pred_scaled = model.predict(X_scaled[val_ids], verbose=0)
    val_pred = y_scaler.inverse_transform(val_pred_scaled).reshape(-1)
    y_val = sample_y[val_ids]
    prev_close_val = sample_prev_close[val_ids]

    gru_metrics = _metrics(y_val, val_pred, prev_close_val)
    zero_pred = np.zeros_like(y_val)
    last_return_pred = sample_last_return[val_ids]
    training_mean_pred = np.full(len(val_ids), np.mean(sample_y[train_ids
```

```
    zero_metrics = _metrics(y_val, zero_pred, prev_close_val)
    last_return_metrics = _metrics(y_val, last_return_pred, prev_close_va
    mean_metrics = _metrics(y_val, training_mean_pred, prev_close_val)

    lookback_rows.append({
        "Lookback (sessions)": lookback,
        "Best epoch": int(np.argmin(history.history["val_loss"]) + 1),
        "Validation samples": int(len(val_ids)),
        "GRU Return MAE (pp)": gru_metrics["Return MAE (pp)"],
        "GRU Return RMSE (pp)": gru_metrics["Return RMSE (pp)"],
        "GRU Close RMSE (points)": gru_metrics["Close RMSE (points)"],
        "GRU Direction accuracy (%)": gru_metrics["Direction accuracy (%)
        "Zero-return RMSE (pp)": zero_metrics["Return RMSE (pp)"],
        "Last-return RMSE (pp)": last_return_metrics["Return RMSE (pp)"],
        "Training-mean RMSE (pp)": mean_metrics["Return RMSE (pp)"],
    })
    lookback_validation_preds[lookback] = {
        "dates": sample_dates[val_ids],
        "actual": y_val,
        "gru": val_pred,
        "zero": zero_pred,
        "last_return": last_return_pred,
        "mean": training_mean_pred,
    }
    del model, history, X_scaled

lookback_df = pd.DataFrame(lookback_rows).sort_values("GRU Return RMSE (p
best_lookback = int(lookback_df.iloc[0]["Lookback (sessions)"])
print("\nValidation-only lookback comparison (lower error is better):")
display(lookback_df.round(4))
print(f"Selected lookback from validation period only: {best_lookback} se

fig, ax = plt.subplots(figsize=(10, 4.5))
ax.plot(lookback_df["Lookback (sessions)"], lookback_df["GRU Return RMSE
ax.plot(lookback_df["Lookback (sessions)"], lookback_df["Zero-return RMSE
ax.plot(lookback_df["Lookback (sessions)"], lookback_df["Last-return RMSE
ax.set_title("Validation return RMSE by lookback window")
ax.set_xlabel("Lookback window (trading sessions)")
ax.set_ylabel("Return RMSE (percentage points)")
ax.grid(alpha=0.25)
ax.legend()
plt.tight_layout()
plt.show()
```

```
[Lookback sweep] Training GRU with lookback=20 ...

[Lookback sweep] Training GRU with lookback=30 ...

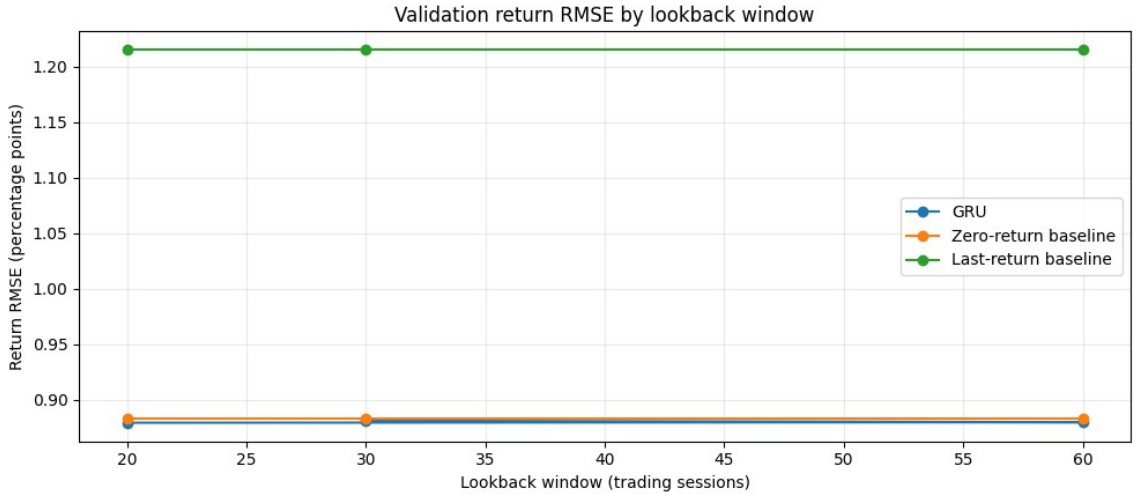
[Lookback sweep] Training GRU with lookback=60 ...

Validation-only lookback comparison (lower error is better):
```

Lookback (sessions)	Best epoch	Validation samples	GRU Return MAE	GRU Return RMSE	GRU Close RMSE	GRU Direction accuracy	Zer retu

	(sessions)	epoch	samples	MAE (pp)	RMSE (pp)	RMSE (points)	accuracy (%)	R (p
0	20	5	692	0.6583	0.8791	150.4340	53.7572	0.8
1	60	2	692	0.6567	0.8796	150.5949	55.4913	0.8
2	30	3	692	0.6614	0.8810	150.8062	54.7688	0.8

```
Selected lookback from validation period only: 20 sessions
```



8. Experiment 2B — Expanding-window walk-forward evaluation

Each fold is forecast using only observations earlier than the fold. The later folds may train on outcomes observed during earlier folds, reflecting an expanding-window evaluation.

```
# ------------------------ 2. Expanding-window walk-forward test ---------
wf_ends, wf_dates, wf_y, wf_prev_close, wf_last_return = _sample_arrays(bes
test_ids_all = np.flatnonzero(wf_dates >= test_start_date)
if len(test_ids_all) < N_WALK_FORWARD_FOLDS * 20:
    raise ValueError(
        f"Not enough held-out test samples ({len(test_ids_all)}) for "
        f"{N_WALK_FORWARD_FOLDS} walk-forward folds."
    )
fold_indices = [chunk for chunk in np.array_split(test_ids_all, N_WALK_FORW

walk_forward_parts = []
fold_metric_rows = []
```

```
print("\n" + "=" * 76)
print("WALK-FORWARD EVALUATION ON THE ORIGINAL HELD-OUT TEST PERIOD")
print("=" * 76)

for fold_number, fold_test_ids in enumerate(fold_indices, start=1):
    fold_start_date = wf_dates[fold_test_ids[0]]
    fold_end_date = wf_dates[fold_test_ids[-1]]
    prior_ids = np.flatnonzero(wf_dates < fold_start_date)
    inner_val_count = max(30, int(len(prior_ids) * VALIDATION_FRACTION_EXP2
    inner_val_count = min(inner_val_count, max(20, len(prior_ids) // 4))
    train_ids = prior_ids[:-inner_val_count]
    inner_val_ids = prior_ids[-inner_val_count:]

    print(
        f"\nFold {fold_number}/{len(fold_indices)}: "
        f"test {pd.Timestamp(fold_start_date).date()} to {pd.Timestamp(fold
        f"train={len(train_ids):,}, inner validation={len(inner_val_ids):,}
        f"test={len(fold_test_ids):,}"
    )

    model, history, X_fold, y_scaler_fold = _fit_gru_for_fold(
        best_lookback, wf_ends, wf_dates, wf_y, train_ids, inner_val_ids
    )
    pred_scaled = model.predict(X_fold[fold_test_ids], verbose=0)
    gru_pred = y_scaler_fold.inverse_transform(pred_scaled).reshape(-1)

    # Baselines use only labels known before this fold's first forecast dat
    zero_pred = np.zeros(len(fold_test_ids), dtype=np.float64)
    last_return_pred = wf_last_return[fold_test_ids].astype(np.float64)
    historical_mean = float(np.mean(wf_y[prior_ids]))
    mean_pred = np.full(len(fold_test_ids), historical_mean, dtype=np.float

    y_fold = wf_y[fold_test_ids].astype(np.float64)
    prev_fold = wf_prev_close[fold_test_ids]
    fold_dates = wf_dates[fold_test_ids]
    fold_predictions = {
        "GRU": gru_pred,
        "Zero-return baseline": zero_pred,
        "Last-return persistence": last_return_pred,
        "Historical-mean baseline": mean_pred,
    }

    part = pd.DataFrame({
        "Fold": fold_number,
        "Target date": fold_dates,
        "Actual log return": y_fold,
        "Previous close": prev_fold,
        "Actual next close": prev_fold * np.exp(y_fold),
    })
    for model_name, pred_return in fold_predictions.items():
        safe_name = model_name
        part[f"{safe_name} predicted return"] = pred_return
        part[f"{safe_name} predicted close"] = prev_fold * np.exp(pred_retu
        metrics = _metrics(
```

```
            y_fold, pred_return, prev_fold,
            include_direction=(model_name != "Zero-return baseline"),
        )
        fold_metric_rows.append({
            "Fold": fold_number,
            "Test start": pd.Timestamp(fold_start_date).date().isoformat(),
            "Test end": pd.Timestamp(fold_end_date).date().isoformat(),
            "Model": model_name,
            **metrics,
        })
    walk_forward_parts.append(part)
    del model, history, X_fold

walk_forward_predictions = pd.concat(walk_forward_parts, ignore_index=True)
fold_metrics_df = pd.DataFrame(fold_metric_rows)

# Aggregate metrics are calculated over all sequentially predicted test blo
aggregate_rows = []
actual_all = walk_forward_predictions["Actual log return"].to_numpy(dtype=r
previous_close_all = walk_forward_predictions["Previous close"].to_numpy(dt
for model_name in [
    "GRU", "Zero-return baseline", "Last-return persistence", "Historical-m
]:
    pred_all = walk_forward_predictions[f"{model_name} predicted return"].t
    aggregate_rows.append({
        "Model": model_name,
        **_metrics(
            actual_all, pred_all, previous_close_all,
            include_direction=(model_name != "Zero-return baseline"),
        ),
    })
walk_forward_summary_df = (
    pd.DataFrame(aggregate_rows)
    .sort_values("Return RMSE (pp)")
    .reset_index(drop=True)
)

print("\nAggregate walk-forward results across all test folds:")
display(walk_forward_summary_df.round(4))
print("\nPer-fold results:")
display(fold_metrics_df.round(4))

fig, axes = plt.subplots(1, 2, figsize=(13, 4.8))
summary_plot = walk_forward_summary_df.set_index("Model")
summary_plot["Return RMSE (pp)"].sort_values().plot(kind="bar", ax=axes[0])
axes[0].set_title("Walk-forward return RMSE")
axes[0].set_ylabel("Return RMSE (percentage points)")
axes[0].set_xlabel("")
axes[0].tick_params(axis="x", rotation=25)
axes[0].grid(axis="y", alpha=0.25)
summary_plot["Direction accuracy (%)"].sort_values(ascending=False).plot(ki
axes[1].axhline(50, linestyle="--", linewidth=1, label="50% reference")
axes[1].set_title("Walk-forward direction accuracy")
axes[1].set_ylabel("Correct direction (%)")
```

```
axes[1].set_xlabel("")
axes[1].tick_params(axis="x", rotation=25)
axes[1].grid(axis="y", alpha=0.25)
axes[1].legend()
plt.tight_layout()
plt.show()
```

```
================================================================================
WALK-FORWARD EVALUATION ON THE ORIGINAL HELD-OUT TEST PERIOD
================================================================================

Fold 1/3: test 2024-01-09 to 2024-12-17 | train=3,340, inner validation=58

Fold 2/3: test 2024-12-18 to 2025-11-21 | train=3,536, inner validation=62

Fold 3/3: test 2025-11-24 to 2026-10-29 | train=3,733, inner validation=65

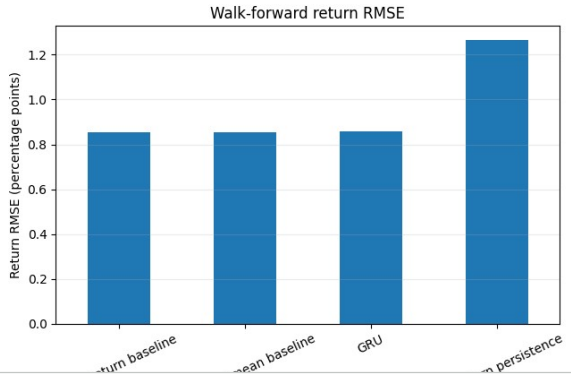
Aggregate walk-forward results across all test folds:
```

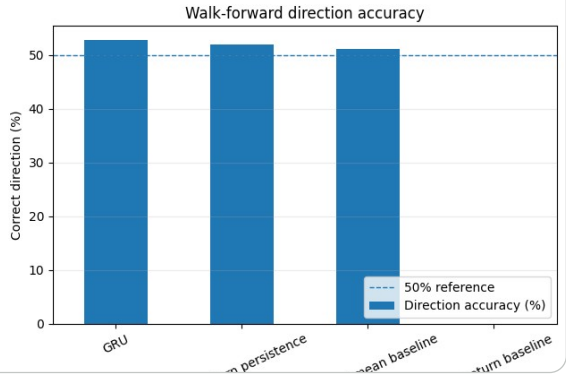
	Model	Return MAE (pp)	Return RMSE (pp)	Close MAE (points)	Close RMSE (points)	Direction accuracy (%)
0	Zero-return baseline	0.6117	0.8541	146.0125	201.9842	NaN
1	Historical-mean baseline	0.6118	0.8544	146.0656	202.1068	51.1561
2	GRU	0.6131	0.8564	146.4584	202.6914	52.7457
3	Last-return persistence	0.8867	1.2644	211.1193	297.1003	51.8786

Per-fold results:

	Fold	Test start	Test end	Model	Return MAE (pp)	Return RMSE (pp)	Close MAE (points)	Cl R (poin
0	1	2024-01-09	2024-12-17	GRU	0.6063	0.9054	141.8854	210.5
1	1	2024-01-09	2024-12-17	Zero-return baseline	0.6061	0.9010	141.6745	209.4
2	1	2024-01-09	2024-12-17	Last-return persistence	0.9243	1.3894	215.3220	318.5
3	1	2024-01-09	2024-12-17	Historical-mean baseline	0.6037	0.8997	141.1469	209.1
4	2	2024-12-18	2025-11-21	GRU	0.5660	0.7643	136.7807	182.7
5	2	2024-12-18	2025-11-21	Zero-return baseline	0.5664	0.7663	136.8769	183.1
6	2	2024-12-18	2025-11-21	Last-return persistence	0.7846	1.0745	189.4705	257.5

7	2	2024-12-18	2025-11-21	Historical-mean baseline	0.5664	0.7657	136.8881	183.0
8	3	2025-11-24	2026-10-29	GRU	0.6671	0.8924	160.7709	213.4
9	3	2025-11-24	2026-10-29	Zero-return baseline	0.6629	0.8885	159.5448	212.1
10	3	2025-11-24	2026-10-29	Last-return persistence	0.9516	1.3084	228.6414	311.4
11	3	2025-11-24	2026-10-29	Historical-mean baseline	0.6655	0.8913	160.2229	212.8





9. Save results for the academic report and GitHub

```
  # ------------------------- 3. Save outputs for the report/GitHub -----
OUTPUT_DIR_EXP2 = Path("outputs")
OUTPUT_DIR_EXP2.mkdir(parents=True, exist_ok=True)

lookback_path = OUTPUT_DIR_EXP2 / "exp2_lookback_validation.csv"
fold_path = OUTPUT_DIR_EXP2 / "exp2_walk_forward_fold_metrics.csv"
summary_path = OUTPUT_DIR_EXP2 / "exp2_walk_forward_summary.csv"
predictions_path = OUTPUT_DIR_EXP2 / "exp2_walk_forward_predictions.csv"
config_path = OUTPUT_DIR_EXP2 / "exp2_config.json"

lookback_df.to_csv(lookback_path, index=False)
fold_metrics_df.to_csv(fold_path, index=False)
walk_forward_summary_df.to_csv(summary_path, index=False)
walk_forward_predictions.to_csv(predictions_path, index=False)
config = {
    "ticker_context": "NIFTY 50 (^NSEI), as downloaded in the original no
    "validation_start_date": str(validation_start_date),
    "held_out_test_start_date": str(test_start_date),
    "lookback_candidates": LOOKBACK_CANDIDATES,
    "selected_lookback_by_validation_return_rmse": best_lookback,
    "walk_forward_folds": N_WALK_FORWARD_FOLDS,
    "max_epochs_per_fit": MAX_EPOCHS_EXP2,
    "batch_size": BATCH_EXP2,
    "random_seed": SEED_EXP2,
    "note": "Walk-forward fold metrics are sequential out-of-sample forec
```

```
}
config_path.write_text(json.dumps(config, indent=2), encoding="utf-8")

print("\nSaved Experiment 2 outputs:")
for path in [lookback_path, fold_path, summary_path, predictions_path, co
    print(f"- {path}")

print("\nInterpretation guardrails:")
print("1. Choose lookback using validation metrics only; do not select it
print("2. GRU is useful only if it improves on sensible baselines consist
print("3. Direction accuracy above 50% is not automatically statistically
print("4. The walk-forward evaluation is historical research, not financi
```

```
Saved Experiment 2 outputs:
- outputs/exp2_lookback_validation.csv
- outputs/exp2_walk_forward_fold_metrics.csv
- outputs/exp2_walk_forward_summary.csv
- outputs/exp2_walk_forward_predictions.csv
- outputs/exp2_config.json

Interpretation guardrails:
1. Choose lookback using validation metrics only; do not select it using t
2. GRU is useful only if it improves on sensible baselines consistently, n
3. Direction accuracy above 50% is not automatically statistically signifi
4. The walk-forward evaluation is historical research, not financial advic
```

Experiment 3 — Feature Engineering and Ablation for GRU Forecasting

Project: Financial Time-Series Forecasting with Recurrent Neural Networks
Dataset: NIFTY 50 daily OHLC data from Yahoo Finance (`^NSEI`)
Model: Same GRU architecture for each feature set

Research question

Do momentum and rolling-volatility features improve next-session return forecasting compared with using daily returns alone?

We compare three feature sets:

- **A. Returns only**
- **B. Returns + momentum** (5- and 10-session)
- **C. Returns + momentum + volatility** (5- and 20-session rolling standard deviation)

This experiment uses the development training/validation periods only. It deliberately does **not** score the previously examined 2024–2026 test period. The goal is to select candidate inputs without repeatedly tuning on that earlier test

period.

All code is visible in notebook cells. Run the cells from top to bottom.

1. Install dependencies

```
%pip -q install yfinance tensorflow scikit-learn matplotlib pandas numpy
```

2. Import libraries and configure data

```
import warnings
warnings.filterwarnings("ignore", category=FutureWarning)

import json
from pathlib import Path

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf
import tensorflow as tf

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
from IPython.display import display

SEED = 42
np.random.seed(SEED)
tf.keras.utils.set_random_seed(SEED)

TICKER = "^NSEI"       # NIFTY 50 index on Yahoo Finance
PERIOD = "max"         # Download available daily history
LOOKBACK = 30          # Baseline window used to derive chronological dat
TRAIN_FRACTION = 0.70
VALIDATION_FRACTION = 0.15
BATCH_SIZE = 32

print("TensorFlow version:", tf.__version__)
print("Ticker:", TICKER, "| Base lookback:", LOOKBACK, "trading sessions"
```

```
TensorFlow version: 2.21.0
Ticker: ^NSEI | Base lookback: 30 trading sessions
```

3. Download and inspect NIFTY 50 data

```
raw = yf.Ticker(TICKER).history(
    period=PERIOD,
```

```
    interval="1d",
    auto_adjust=False,
    actions=False,
)

if raw is None or raw.empty:
    raise RuntimeError(
        "No data was returned. Check your internet connection, ticker sym
        "or Yahoo Finance availability, then retry this cell."
    )

required_columns = ["Open", "High", "Low", "Close"]
missing = [col for col in required_columns if col not in raw.columns]
if missing:
    raise ValueError(f"Downloaded data is missing required columns: {miss

data = raw[required_columns].copy()
data = data.apply(pd.to_numeric, errors="coerce")
data = data.dropna(subset=required_columns)
data = data[~data.index.duplicated(keep="last")].sort_index()

if len(data) < 500:
    raise ValueError(
        f"Only {len(data)} rows downloaded; more history is needed for a
    )

print(f"Rows downloaded: {len(data):,}")
print("Date range:", data.index.min().date(), "to", data.index.max().date
display(data.head())
display(data.tail())

fig, ax = plt.subplots(figsize=(13, 4))
ax.plot(data.index, data["Close"], linewidth=1.2)
ax.set_title(f"{TICKER} — Daily Closing Level")
ax.set_xlabel("Date")
ax.set_ylabel("Index level")
ax.grid(alpha=0.25)
plt.tight_layout()
plt.show()
```

```
Rows downloaded: 4,690
Date range: 2007-09-17 to 2026-10-29
```

Date	Open	High	Low	Close
2007-09-17 00:00:00+05:30	4518.450195	4549.049805	4482.850098	4494.649902
2007-09-18 00:00:00+05:30	4494.100098	4551.799805	4481.549805	4546.200195
2007-09-19 00:00:00+05:30	4550.250000	4739.000000	4550.250000	4732.350098
2007-09-20	4734.850098	4760.850098	4721.149902	4747.549805

00:00:00+05:30				
2007-09-21 00:00:00+05:30	4752.950195	4855.700195	4733.700195	4837.549805

Date	Open	High	Low	Close
2026-10-23 00:00:00+05:30	22742.199219	22787.250000	22372.000000	22600.000000
2026-10-26 00:00:00+05:30	22524.400391	22753.199219	22387.349609	22753.199219
2026-10-27 00:00:00+05:30	22708.800781	22829.750000	22636.849609	22749.750000
2026-10-28 00:00:00+05:30	22678.000000	22778.050781	22497.699219	22698.849609
2026-10-29 00:00:00+05:30	22690.650391	22874.150391	22656.750000	22846.000000



4. Build features and next-session return target

Every rolling feature is calculated using only current or past rows. Future next-day return is stored separately as the target.

```
# Calculate features using information available on or before each foreca
close = data["Close"]
log_return = np.log(close).diff()

features = pd.DataFrame(index=data.index)
features["log_return_1d"] = log_return
features["intraday_range"] = (data["High"] - data["Low"]) / close
features["open_close_return"] = (close - data["Open"]) / data["Open"]
features["momentum_5d"] = close.pct_change(5)
features["momentum_10d"] = close.pct_change(10)
features["volatility_5d"] = log_return.rolling(5).std()
features["volatility_20d"] = log_return.rolling(20).std()
features["sma_gap_10d"] = close / close.rolling(10).mean() - 1.0
features["sma_gap_20d"] = close / close.rolling(20).mean() - 1.0
features["sma_gap_50d"] = close / close.rolling(50).mean() - 1.0

FEATURE_COLUMNS = list(features.columns)
```

```
next_day_log_return = log_return.shift(-1)
next_day_date = pd.Series(data.index, index=data.index).shift(-1)

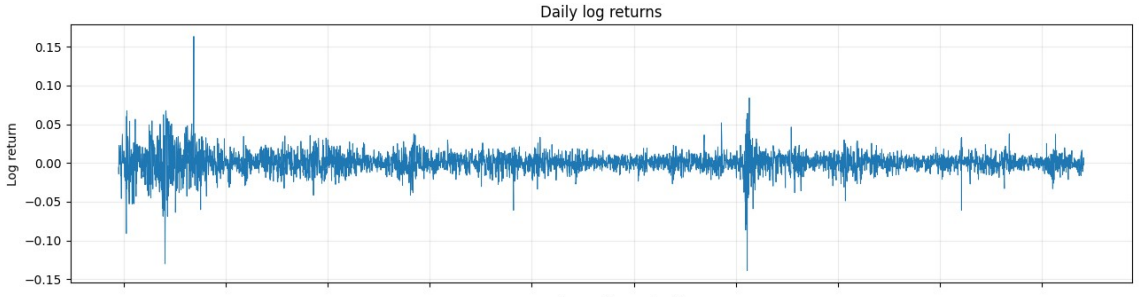
model_df = features.copy()
model_df["target_return"] = next_day_log_return
model_df["target_date"] = next_day_date
model_df["close_price"] = close
model_df = model_df.replace([np.inf, -np.inf], np.nan)
model_df = model_df.dropna(
    subset=FEATURE_COLUMNS + ["target_return", "target_date", "close_pric
)

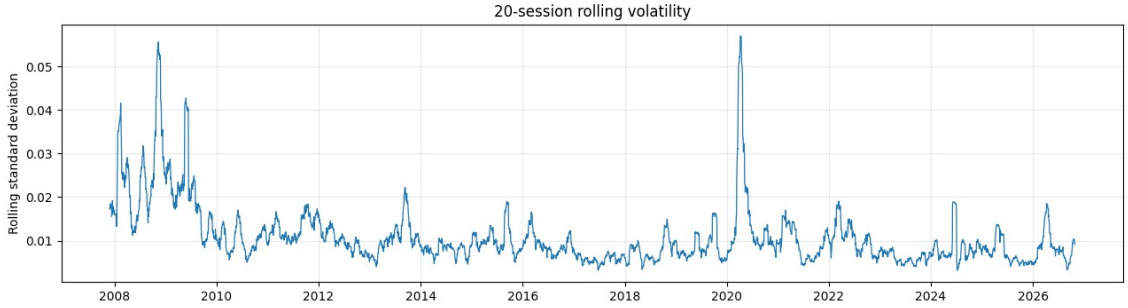
print("Features:", FEATURE_COLUMNS)
print("Usable feature rows:", f"{len(model_df):,}")
display(model_df[FEATURE_COLUMNS + ["target_return", "target_date"]].head

fig, axes = plt.subplots(2, 1, figsize=(13, 7), sharex=True)
axes[0].plot(model_df.index, model_df["log_return_1d"], linewidth=0.7)
axes[0].set_title("Daily log returns")
axes[0].set_ylabel("Log return")
axes[0].grid(alpha=0.25)
axes[1].plot(model_df.index, model_df["volatility_20d"], linewidth=0.9)
axes[1].set_title("20-session rolling volatility")
axes[1].set_ylabel("Rolling standard deviation")
axes[1].grid(alpha=0.25)
plt.tight_layout()
plt.show()
```

```
Features: ['log_return_1d', 'intraday_range', 'open_close_return', 'moment
Usable feature rows: 4,640
```

Date	log_return_1d	intraday_range	open_close_return	momentum_
2007-11-27 00:00:00+05:30	-0.005871	0.015435	-0.005428	-0.0143
2007-11-28 00:00:00+05:30	-0.014246	0.027494	-0.014387	0.0101
2007-11-29 00:00:00+05:30	0.003031	0.020037	0.002991	0.0208
2007-11-30 00:00:00+05:30	0.022489	0.026012	0.022870	0.0274
2007-12-03 00:00:00+05:30	0.017588	0.021176	0.017267	0.0232





5. Configure Experiment 3 and establish the development split

The default lookback is 20 sessions, based on Experiment 2's validation-only selection. Change `LOOKBACK_EXP3` only if you can justify that selection using development validation data. Do not select features based on the previously inspected test period.

```
from pathlib import Path
import json
import warnings

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import tensorflow as tf
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
from IPython.display import display

warnings.filterwarnings("ignore", category=FutureWarning)

# -------------------------- 0. Check notebook context ------------------
_required_exp3 = ["model_df"]
```

```
_missing_exp3 = [name for name in _required_exp3 if name not in globals()]
if _missing_exp3:
    raise RuntimeError(
        "Missing required variable(s): " + ", ".join(_missing_exp3)
        + ". Run the original notebook through its feature-engineering ce
        "then execute this file using `%run -i financial_timeseries_exper
    )
if not isinstance(model_df, pd.DataFrame):
    raise TypeError("`model_df` must be the DataFrame created by the orig

_exp3_feature_sets = {
    "Returns only": ["log_return_1d"],
    "Returns + momentum": ["log_return_1d", "momentum_5d", "momentum_10d"
    "Returns + momentum + volatility": [
        "log_return_1d", "momentum_5d", "momentum_10d", "volatility_5d",
    ],
}
_exp3_required = {
    "target_return", "target_date", "close_price", *[c for cols in _exp3_
}
_exp3_missing = sorted(_exp3_required - set(model_df.columns))
if _exp3_missing:
    raise ValueError(
        "`model_df` is missing columns needed by Experiment 3: " + ", ".j
        + ". Check that the original notebook's feature-engineering cell
    )

SEED_EXP3 = int(globals().get("SEED", 42))
BATCH_EXP3 = int(globals().get("BATCH_SIZE", 32))
LOOKBACK_EXP3 = int(globals().get("best_lookback", 20))
MAX_EPOCHS_EXP3 = 25
PATIENCE_EXP3 = 5
TRAIN_FRACTION_EXP3 = float(globals().get("TRAIN_FRACTION", 0.70))
VALIDATION_FRACTION_EXP3 = float(globals().get("VALIDATION_FRACTION", 0.1

np.random.seed(SEED_EXP3)
tf.keras.utils.set_random_seed(SEED_EXP3)

_exp3_n_rows = len(model_df)
if _exp3_n_rows < 500:
    raise ValueError(f"Only {_exp3_n_rows} rows are available; at least 5
if not 1 <= LOOKBACK_EXP3 < _exp3_n_rows:
    raise ValueError(f"Invalid lookback: {LOOKBACK_EXP3}")

_exp3_target_dates_all = pd.DatetimeIndex(pd.to_datetime(model_df["target_
_exp3_target_return_all = model_df["target_return"].to_numpy(dtype=np.flo
_exp3_previous_close_all = model_df["close_price"].to_numpy(dtype=np.floa

# Match Experiment 2's train/validation/test boundaries, but only use the
# two periods here. The historical test period is deliberately excluded.
_exp3_original_lookback = int(globals().get("LOOKBACK", 30))
_exp3_base_ends = np.arange(_exp3_original_lookback - 1, _exp3_n_rows)
_exp3_base_dates = _exp3_target_dates_all[_exp3_base_ends]
_exp3_base_n = len(_exp3_base_ends)
_exp3_train_end = int(_exp3_base_n * TRAIN_FRACTION_EXP3)
```

```
_exp3_train_end = int(_exp3_base_n * TRAIN_FRACTION_EXP3)
_exp3_val_end = int(_exp3_base_n * (TRAIN_FRACTION_EXP3 + VALIDATION_FRAC
if _exp3_train_end < 100 or _exp3_val_end <= _exp3_train_end or _exp3_val
    raise ValueError("Invalid chronological split sizes; check the config
_exp3_validation_start = _exp3_base_dates[_exp3_train_end]
_exp3_old_test_start = _exp3_base_dates[_exp3_val_end]

print("=" * 78)
print("EXPERIMENT 3 — FEATURE-SET ABLATION ON DEVELOPMENT VALIDATION DATA
print("=" * 78)
print(f"Rows available: {_exp3_n_rows:,}")
print(f"Lookback sessions: {LOOKBACK_EXP3}")
print(f"Training targets before: {_exp3_validation_start}")
print(f"Validation targets: {_exp3_validation_start} to before {_exp3_old
print("Previously inspected historical test period is NOT evaluated in th
print("Feature sets:")
for _exp3_name, _exp3_cols in _exp3_feature_sets.items():
    print(f"  - {_exp3_name}: {_exp3_cols}")

# Generate sample endpoints once. All feature sets use the same rows and
_exp3_sample_ends = np.arange(LOOKBACK_EXP3 - 1, _exp3_n_rows)
_exp3_sample_dates = _exp3_target_dates_all[_exp3_sample_ends]
_exp3_y = _exp3_target_return_all[_exp3_sample_ends]
_exp3_prev_close = _exp3_previous_close_all[_exp3_sample_ends]
_exp3_train_ids = np.flatnonzero(_exp3_sample_dates < _exp3_validation_st
_exp3_val_ids = np.flatnonzero(
    (_exp3_sample_dates >= _exp3_validation_start) & (_exp3_sample_dates
)
if len(_exp3_train_ids) < 100 or len(_exp3_val_ids) < 50:
    raise ValueError(
        f"Not enough samples for the development split: train={len(_exp3_
        f"validation={len(_exp3_val_ids)}."
    )
```

```
==============================================================================
EXPERIMENT 3 — FEATURE-SET ABLATION ON DEVELOPMENT VALIDATION DATA
==============================================================================
Rows available: 4,640
Lookback sessions: 20
Training targets before: 2021-03-23 00:00:00+05:30
Validation targets: 2021-03-23 00:00:00+05:30 to before 2024-01-09 00:00:0
Previously inspected historical test period is NOT evaluated in this exper
Feature sets:
  - Returns only: ['log_return_1d']
  - Returns + momentum: ['log_return_1d', 'momentum_5d', 'momentum_10d']
  - Returns + momentum + volatility: ['log_return_1d', 'momentum_5d', 'mom
```

6. Define sequences, GRU architecture, training callbacks, and evaluation metrics

All feature sets use the same model architecture, training settings, dates, and target. The feature scaler and target scaler are fitted using training data only.

```
def _exp3_make_sequences(scaled_features, sample_ends, lookback):
    """Build fixed-length trailing sequences; each uses only rows through
    return np.stack(
        [scaled_features[end - lookback + 1 : end + 1] for end in sample_
    ).astype(np.float32)


def _exp3_build_gru(input_shape):
    """Identical GRU architecture for every feature set."""
    model = tf.keras.Sequential(name="gru_feature_ablation_exp3")
    model.add(tf.keras.layers.Input(shape=input_shape))
    model.add(tf.keras.layers.GRU(64, return_sequences=True))
    model.add(tf.keras.layers.Dropout(0.2))
    model.add(tf.keras.layers.GRU(32))
    model.add(tf.keras.layers.Dense(16, activation="relu"))
    model.add(tf.keras.layers.Dense(1, activation="linear"))
    model.compile(
        optimizer=tf.keras.optimizers.Adam(learning_rate=1e-3),
        loss=tf.keras.losses.Huber(),
        metrics=[tf.keras.metrics.MeanAbsoluteError(name="mae")],
    )
    return model


def _exp3_callbacks():
    return [
        tf.keras.callbacks.EarlyStopping(
            monitor="val_loss", patience=PATIENCE_EXP3,
            restore_best_weights=True, verbose=0
        ),
        tf.keras.callbacks.ReduceLROnPlateau(
            monitor="val_loss", factor=0.5, patience=3,
            min_lr=1e-5, verbose=0
        ),
    ]


def _exp3_metrics(y_true, y_pred, previous_close):
    y_true = np.asarray(y_true, dtype=np.float64).reshape(-1)
    y_pred = np.asarray(y_pred, dtype=np.float64).reshape(-1)
    previous_close = np.asarray(previous_close, dtype=np.float64).reshape
    actual_close = previous_close * np.exp(y_true)
    predicted_close = previous_close * np.exp(y_pred)
    return {
        "Return MAE (pp)": mean_absolute_error(y_true, y_pred) * 100,
        "Return RMSE (pp)": np.sqrt(mean_squared_error(y_true, y_pred)) *
        "Close MAE (points)": mean_absolute_error(actual_close, predicted
        "Close RMSE (points)": np.sqrt(mean_squared_error(actual_close, p
        "Direction accuracy (%)": np.mean(np.sign(y_true) == np.sign(y_pr
    }
```

7. Train the three feature-set experiments

7. Train the three feature-set experiments

This takes longer than a simple inference cell because the same GRU architecture is trained three times.

```
# ------------------------- 1. Fit feature ablations ------------------
_exp3_result_rows = []
_exp3_val_prediction_frames = []
_exp3_saved_models = {}

for _exp3_feature_name, _exp3_columns in _exp3_feature_sets.items():
    print("\n" + "-" * 76)
    print(f"Training GRU: {_exp3_feature_name}")
    print("Input columns:", _exp3_columns)

    _exp3_raw_features = model_df[_exp3_columns].to_numpy(dtype=np.float3

    # Fit input scaler using feature rows only through the last training
    _exp3_last_train_origin = int(_exp3_sample_ends[_exp3_train_ids[-1]])
    _exp3_x_scaler = StandardScaler()
    _exp3_x_scaler.fit(_exp3_raw_features[: _exp3_last_train_origin + 1])
    _exp3_scaled_features = _exp3_x_scaler.transform(_exp3_raw_features).
    _exp3_X = _exp3_make_sequences(_exp3_scaled_features, _exp3_sample_en

    # Fit target scaler on training targets only.
    _exp3_y_scaler = StandardScaler()
    _exp3_y_scaler.fit(_exp3_y[_exp3_train_ids].reshape(-1, 1))
    _exp3_y_scaled = _exp3_y_scaler.transform(_exp3_y.reshape(-1, 1)).ast

    tf.keras.backend.clear_session()
    tf.keras.utils.set_random_seed(SEED_EXP3)
    _exp3_model = _exp3_build_gru((LOOKBACK_EXP3, len(_exp3_columns)))
    _exp3_history = _exp3_model.fit(
        _exp3_X[_exp3_train_ids], _exp3_y_scaled[_exp3_train_ids],
        validation_data=(_exp3_X[_exp3_val_ids], _exp3_y_scaled[_exp3_val
        epochs=MAX_EPOCHS_EXP3,
        batch_size=BATCH_EXP3,
        shuffle=False,
        callbacks=_exp3_callbacks(),
        verbose=0,
    )

    _exp3_val_scaled_pred = _exp3_model.predict(_exp3_X[_exp3_val_ids], v
    _exp3_val_pred = _exp3_y_scaler.inverse_transform(_exp3_val_scaled_pr
    _exp3_val_actual = _exp3_y[_exp3_val_ids]
    _exp3_val_close = _exp3_prev_close[_exp3_val_ids]
    _exp3_gru_metrics = _exp3_metrics(_exp3_val_actual, _exp3_val_pred, _

    # Baselines are calculated on the exact same validation rows.
    _exp3_zero = np.zeros_like(_exp3_val_actual)
    _exp3_training_mean = np.full(
        len(_exp3_val_ids), float(np.mean(_exp3_y[_exp3_train_ids])), dty
    )
```

```
    _exp3_last_return = model_df["log_return_1d"].to_numpy(dtype=np.float
    _exp3_zero_metrics = _exp3_metrics(_exp3_val_actual, _exp3_zero, _exp
    _exp3_mean_metrics = _exp3_metrics(_exp3_val_actual, _exp3_training_m
    _exp3_last_metrics = _exp3_metrics(_exp3_val_actual, _exp3_last_retur

    _exp3_result_rows.append({
        "Feature set": _exp3_feature_name,
        "Feature count": len(_exp3_columns),
        "Best epoch": int(np.argmin(_exp3_history.history["val_loss"]) +
        "Validation samples": int(len(_exp3_val_ids)),
        "GRU Return MAE (pp)": _exp3_gru_metrics["Return MAE (pp)"],
        "GRU Return RMSE (pp)": _exp3_gru_metrics["Return RMSE (pp)"],
        "GRU Close RMSE (points)": _exp3_gru_metrics["Close RMSE (points)
        "GRU Direction accuracy (%)": _exp3_gru_metrics["Direction accura
        "Zero-return RMSE (pp)": _exp3_zero_metrics["Return RMSE (pp)"],
        "Historical-mean RMSE (pp)": _exp3_mean_metrics["Return RMSE (pp)
        "Last-return RMSE (pp)": _exp3_last_metrics["Return RMSE (pp)"],
        "Delta vs zero baseline (pp)": _exp3_gru_metrics["Return RMSE (pp
    })

    _exp3_val_prediction_frames.append(pd.DataFrame({
        "Target date": _exp3_sample_dates[_exp3_val_ids],
        "Feature set": _exp3_feature_name,
        "Actual return": _exp3_val_actual,
        "GRU predicted return": _exp3_val_pred,
        "Zero-return predicted return": _exp3_zero,
        "Historical-mean predicted return": _exp3_training_mean,
        "Last-return predicted return": _exp3_last_return,
        "Previous close": _exp3_val_close,
    }))

    _exp3_saved_models[_exp3_feature_name] = {
        "features": list(_exp3_columns),
        "best_epoch": int(np.argmin(_exp3_history.history["val_loss"]) +
    }
    print(
        f"Best epoch={_exp3_saved_models[_exp3_feature_name]['best_epoch'
        f"Validation return RMSE={_exp3_gru_metrics['Return RMSE (pp)']:.
        f"Zero-return RMSE={_exp3_zero_metrics['Return RMSE (pp)']:.4f} p
    )

    del _exp3_model, _exp3_history, _exp3_X, _exp3_scaled_features
```

```
--------------------------------------------------------------------------------
Training GRU: Returns only
Input columns: ['log_return_1d']
Best epoch=5 | Validation return RMSE=0.8790 pp | Zero-return RMSE=0.8830

--------------------------------------------------------------------------------
Training GRU: Returns + momentum
Input columns: ['log_return_1d', 'momentum_5d', 'momentum_10d']
Best epoch=2 | Validation return RMSE=0.8805 pp | Zero-return RMSE=0.8830

--------------------------------------------------------------------------------
```

```
Training GRU: Returns + momentum + volatility
Input columns: ['log_return_1d', 'momentum_5d', 'momentum_10d', 'volatilit
Best epoch=1 | Validation return RMSE=0.8821 pp | Zero-return RMSE=0.8830
```

8. Compare validation performance

Lower return RMSE is better. Compare each GRU with the zero-return, historical-mean and last-return baselines. Small differences may be noise and should not be overstated.

```
# ------------------------------ 2. Summarize -------------------------
feature_ablation_df = pd.DataFrame(_exp3_result_rows).sort_values(
    "GRU Return RMSE (pp)"
).reset_index(drop=True)
feature_ablation_predictions_df = pd.concat(_exp3_val_prediction_frames,
_best_feature_set_exp3 = str(feature_ablation_df.iloc[0]["Feature set"])
_zero_rmse_exp3 = float(feature_ablation_df.iloc[0]["Zero-return RMSE (pp
_best_rmse_exp3 = float(feature_ablation_df.iloc[0]["GRU Return RMSE (pp)

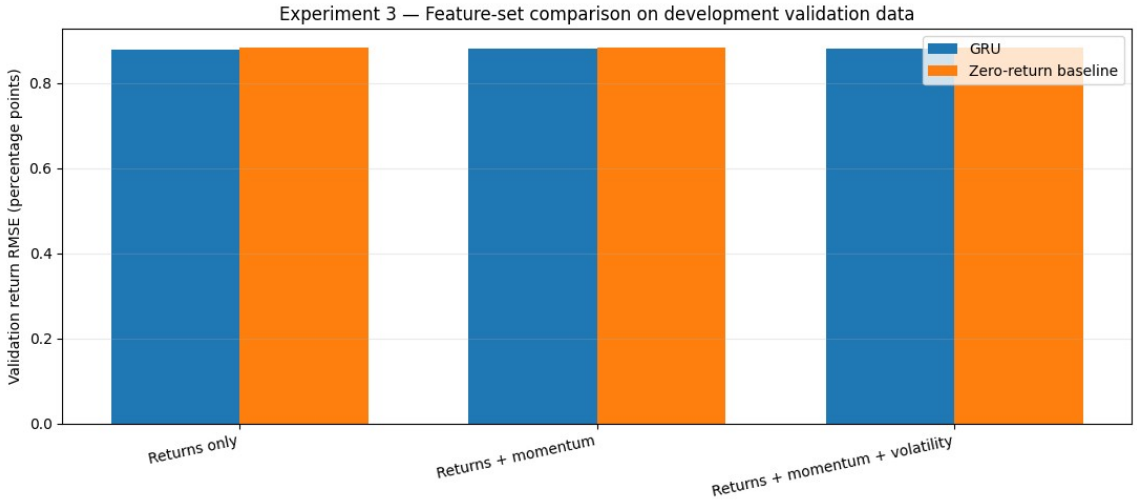
print("\n" + "=" * 78)
print("VALIDATION-ONLY FEATURE-SET COMPARISON (lower RMSE is better)")
print("=" * 78)
display(feature_ablation_df.round(4))
print(f"Lowest GRU validation RMSE feature set: {_best_feature_set_exp3}"
print(f"Best GRU RMSE: {_best_rmse_exp3:.4f} pp")
print(f"Zero-return baseline RMSE: {_zero_rmse_exp3:.4f} pp")
if _best_rmse_exp3 < _zero_rmse_exp3:
    print("Result: the best GRU feature set beats the zero-return baselin
else:
    print("Result: no tested GRU feature set beats the zero-return baseli

fig, ax = plt.subplots(figsize=(11, 5))
_plot_df_exp3 = feature_ablation_df.set_index("Feature set")
_x = np.arange(len(_plot_df_exp3))
_width = 0.36
ax.bar(_x - _width / 2, _plot_df_exp3["GRU Return RMSE (pp)"], _width, la
ax.bar(_x + _width / 2, _plot_df_exp3["Zero-return RMSE (pp)"], _width, l
ax.set_xticks(_x)
ax.set_xticklabels(_plot_df_exp3.index, rotation=12, ha="right")
ax.set_ylabel("Validation return RMSE (percentage points)")
ax.set_title("Experiment 3 — Feature-set comparison on development valida
ax.grid(axis="y", alpha=0.25)
ax.legend()
plt.tight_layout()
plt.show()
```

```
==========================================================================
VALIDATION-ONLY FEATURE-SET COMPARISON (lower RMSE is better)
==========================================================================
                                                    GRU      GRU        GRU         G
```

	Feature set	Feature count	Best epoch	Validation samples	Return MAE (pp)	Return RMSE (pp)	Close RMSE (points)	Directi accura (
0	Returns only	1	5	692	0.6576	0.8790	150.3850	56.35
1	Returns + momentum	3	2	692	0.6586	0.8805	150.6198	54.33
2	Returns + momentum + volatility	5	1	692	0.6600	0.8821	150.9879	54.33

```
Lowest GRU validation RMSE feature set: Returns only
Best GRU RMSE: 0.8790 pp
Zero-return baseline RMSE: 0.8830 pp
Result: the best GRU feature set beats the zero-return baseline on this va
```



9. Save results for the report and GitHub

```
# ------------------------------- 3. Save -------------------------------
_exp3_output_dir = Path("outputs")
_exp3_output_dir.mkdir(parents=True, exist_ok=True)
feature_ablation_df.to_csv(_exp3_output_dir / "exp3_feature_validation_me
feature_ablation_predictions_df.to_csv(
    _exp3_output_dir / "exp3_feature_validation_predictions.csv", index=F
)
_exp3_config = {
```

```
    "experiment": "Experiment 3 — Feature-set ablation",
    "ticker": str(globals().get("TICKER", "^NSEI")),
    "lookback_sessions": LOOKBACK_EXP3,
    "training_targets_before": str(_exp3_validation_start),
    "validation_targets_from": str(_exp3_validation_start),
    "validation_targets_before": str(_exp3_old_test_start),
    "historical_test_evaluated": False,
    "seed": SEED_EXP3,
    "max_epochs": MAX_EPOCHS_EXP3,
    "batch_size": BATCH_EXP3,
    "feature_sets": _exp3_feature_sets,
    "selected_feature_set_by_validation_rmse": _best_feature_set_exp3,
}
with open(_exp3_output_dir / "exp3_config.json", "w", encoding="utf-8") a
    json.dump(_exp3_config, _exp3_f, indent=2)

print("\nSaved Experiment 3 outputs:")
for _exp3_path in sorted(_exp3_output_dir.glob("exp3_*")):
    print(f"- {_exp3_path} ({_exp3_path.stat().st_size / 1024:.1f} KB)")
print("\nInterpretation guardrails:")
print("1. Feature sets are compared on the development validation period
print("2. The previously inspected 2024–2026 test period was not evaluate
print("3. Small validation RMSE differences may be noise; repeatability m
print("4. Better forecast error is not evidence of profitable trading.")
```

```
Saved Experiment 3 outputs:
- outputs/exp3_config.json (0.8 KB)
- outputs/exp3_feature_validation_metrics.csv (0.8 KB)
- outputs/exp3_feature_validation_predictions.csv (259.4 KB)

Interpretation guardrails:
1. Feature sets are compared on the development validation period only.
2. The previously inspected 2024–2026 test period was not evaluated or use
3. Small validation RMSE differences may be noise; repeatability matters.
4. Better forecast error is not evidence of profitable trading.
```

Experiment 4 — External-Market Replication on NIFTY Bank

Research question

Does the selected **returns-only GRU with a 20-session lookback** show similar forecasting behavior on a second market series, NIFTY Bank (`^NSEBANK`), or was the earlier result specific to NIFTY 50?

This notebook uses a fixed feature set and lookback selected in the earlier NIFTY 50 experiments. It does **not** tune those choices on the new index. It trains a fresh model on NIFTY Bank's own earlier data, chooses the training epoch using validation data from 2021–2023, and evaluates on the later 2024-onward test

period.

> This is an external-market replication of the **method**, not transfer learning: the model weights are trained from scratch on NIFTY Bank. Historical forecast evaluation is not financial advice or evidence of a profitable trading strategy.

1. Install / import dependencies

Run cells in order. Google Colab normally includes TensorFlow; `yfinance` may need installation.

```
%pip -q install yfinance scikit-learn
```

```
import os
import json
import random
import warnings
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
import yfinance as yf
import tensorflow as tf

from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_absolute_error, mean_squared_error
from tensorflow.keras import Sequential
from tensorflow.keras.layers import Input, GRU, Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping

warnings.filterwarnings("ignore", category=FutureWarning)
SEED = 42
random.seed(SEED)
np.random.seed(SEED)
tf.random.set_seed(SEED)

print("TensorFlow:", tf.__version__)
print("NumPy:", np.__version__)
print("Pandas:", pd.__version__)
```

```
TensorFlow: 2.21.0
NumPy: 2.1.3
Pandas: 2.2.3
```

2. Experimental configuration

The date boundaries are fixed before looking at the test results. The validation

period is used for early stopping; the test period is held back for final evaluation in this notebook.

```
TICKER = "^NSEBANK"       # NIFTY Bank index on Yahoo Finance
LOOKBACK = 20             # Selected in Experiment 3; keep fixed here
TRAIN_END = pd.Timestamp("2020-12-31")
VAL_START = pd.Timestamp("2021-01-01")
VAL_END = pd.Timestamp("2023-12-31")
TEST_START = pd.Timestamp("2024-01-01")
EPOCHS = 60
BATCH_SIZE = 32

print({
    "ticker": TICKER,
    "lookback_sessions": LOOKBACK,
    "train_targets_through": str(TRAIN_END.date()),
    "validation_targets": f"{VAL_START.date()} to {VAL_END.date()}",
    "test_targets_from": str(TEST_START.date()),
})
```

```
{'ticker': '^NSEBANK', 'lookback_sessions': 20, 'train_targets_through': '
```

3. Download and inspect historical data

We use the daily Close series and compute daily log returns in **percentage points**. The index ticker may return multi-level columns in some `yfinance` versions, so the extraction below handles both common formats.

```
raw = yf.download(
    TICKER,
    period="max",
    interval="1d",
    auto_adjust=True,
    progress=False,
    threads=False,
)

if raw.empty:
    raise RuntimeError(
        f"No data returned for {TICKER}. Check internet access and the Ya
    )

if isinstance(raw.columns, pd.MultiIndex):
    if "Close" in raw.columns.get_level_values(0):
        close_obj = raw["Close"]
        close = close_obj.iloc[:, 0] if isinstance(close_obj, pd.DataFram
    elif "Close" in raw.columns.get_level_values(-1):
        close_obj = raw.xs("Close", axis=1, level=-1)
        close = close_obj.iloc[:, 0] if isinstance(close_obj, pd.DataFram
    else:
```

```
    else:
        raise KeyError("Could not find a Close column in downloaded data.
else:
    close = raw["Close"]

close = pd.to_numeric(close, errors="coerce").dropna().sort_index()
close.index = pd.to_datetime(close.index)
# Remove timezone for consistent date comparisons (the dates represent tr
if getattr(close.index, "tz", None) is not None:
    close.index = close.index.tz_localize(None)
close = close[~close.index.duplicated(keep="last")]
close.name = "Close"

# Return at date t is known after the close at t. The model predicts this
# using only the previous LOOKBACK daily returns.
returns = (100.0 * np.log(close / close.shift(1))).dropna()
returns.name = "LogReturn_pp"

print("Downloaded rows:", len(close))
print("Date range:", close.index.min().date(), "to", close.index.max().da
print("Return observations:", len(returns))
display(pd.DataFrame({"Close": close, "LogReturn_pp": returns}).tail())

if returns.index.max() < TEST_START:
    raise RuntimeError(
        f"Latest data date ({returns.index.max().date()}) is before the p
    )
```

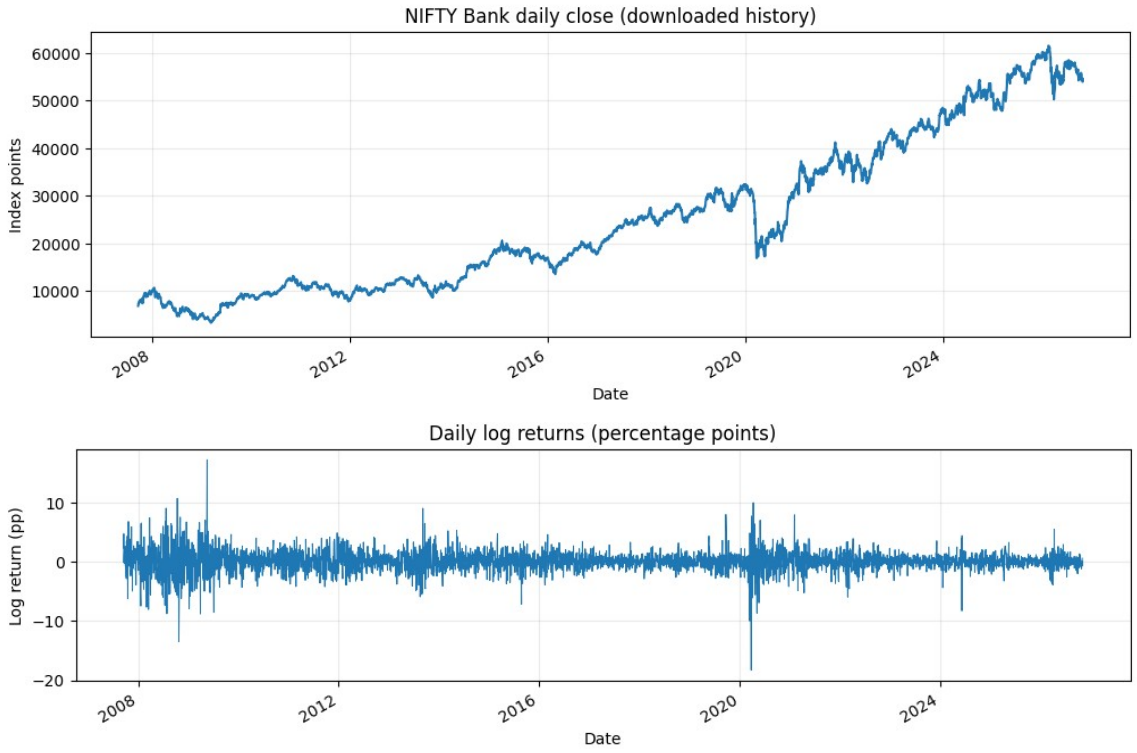
```
Downloaded rows: 4705
Date range: 2007-09-17 to 2026-10-29
Return observations: 4704
```

Date	Close	LogReturn_pp
2026-10-23	54287.351562	-1.169009
2026-10-26	54366.699219	0.146056
2026-10-27	53974.898438	-0.723272
2026-10-28	54267.398438	0.540455
2026-10-29	54587.648438	0.588399

```
fig, ax = plt.subplots(figsize=(12, 4))
close.plot(ax=ax, title="NIFTY Bank daily close (downloaded history)")
ax.set_xlabel("Date")
ax.set_ylabel("Index points")
ax.grid(True, alpha=0.25)
plt.show()

fig, ax = plt.subplots(figsize=(12, 3))
returns.plot(ax=ax, title="Daily log returns (percentage points)", linewi
ax.set_xlabel("Date")
ax.set_ylabel("Log return (pp)")
```

```
ax.set_ylabel("Log return (pp)")
ax.grid(True, alpha=0.25)
plt.show()
```



4. Create one-step sequences and chronological splits

Each sample uses the prior 20 observed daily returns to predict the next session's return. Splits are assigned by the **target date**, so a validation/test sequence may legitimately use earlier observed returns from the previous period as context. No future target returns are included in an input sequence.

```
return_values = returns.to_numpy(dtype=np.float32)
return_dates = returns.index
```

```
X_raw = np.array([
    return_values[i - LOOKBACK:i].reshape(LOOKBACK, 1)
    for i in range(LOOKBACK, len(return_values))
], dtype=np.float32)
y_raw = return_values[LOOKBACK:].reshape(-1, 1)
sample_dates = return_dates[LOOKBACK:]

train_mask = sample_dates <= TRAIN_END
val_mask = (sample_dates >= VAL_START) & (sample_dates <= VAL_END)
test_mask = sample_dates >= TEST_START

if train_mask.sum() == 0 or val_mask.sum() == 0 or test_mask.sum() == 0:
    raise ValueError(
        "A split has no samples. Check the downloaded date range and conf
    )

print("Training samples:", int(train_mask.sum()))
print("Validation samples:", int(val_mask.sum()))
print("Test samples:", int(test_mask.sum()))
print("Validation dates:", sample_dates[val_mask][0].date(), "to", sample
print("Test dates:", sample_dates[test_mask][0].date(), "to", sample_date
```

```
Training samples: 3246
Validation samples: 741
Test samples: 697
Validation dates: 2021-01-01 to 2023-12-29
Test dates: 2024-01-01 to 2026-10-29
```

5. Scale using training data only

For the **initial training/validation fit**, the scaler is fitted only on returns available up to 2020-12-31. That initial scaler transforms the training, validation, and test sequences used to select an epoch count. After the validation step, a later cell fits a fresh scaler on all pre-2024 observations and retrains the final model before evaluating the held-out test period. At no point is the 2024-onward test data used to fit the scaler.

```
scaler = StandardScaler()
train_return_values = returns.loc[returns.index <= TRAIN_END].to_numpy().
scaler.fit(train_return_values)

X_scaled = scaler.transform(X_raw.reshape(-1, 1)).reshape(X_raw.shape).as
y_scaled = scaler.transform(y_raw).astype(np.float32)

X_train, y_train = X_scaled[train_mask], y_scaled[train_mask]
X_val, y_val = X_scaled[val_mask], y_scaled[val_mask]
X_test, y_test = X_scaled[test_mask], y_scaled[test_mask]

y_test_actual_pp = y_raw[test_mask].ravel()
print("X_train shape:", X_train.shape, "y_train shape:", y_train.shape)
print("X_val shape:", X_val.shape, "y_val shape:", y_val.shape)
```

```
print("X_test shape:", X_test.shape, "y_test shape:", y_test.shape)
print("Training scaler mean:", float(scaler.mean_[0]))
print("Training scaler scale:", float(scaler.scale_[0]))
```

```
X_train shape: (3246, 20, 1) y_train shape: (3246, 1)
X_val shape: (741, 20, 1) y_val shape: (741, 1)
X_test shape: (697, 20, 1) y_test shape: (697, 1)
Training scaler mean: 0.046275806589103095
Training scaler scale: 1.9093242498439407
```

6. Build and train the GRU

We keep the experiment intentionally modest: one GRU layer, dropout, and a linear output for next-day log return. Early stopping selects the best epoch using only the validation period.

```
model = Sequential([
    Input(shape=(LOOKBACK, 1)),
    GRU(32),
    Dropout(0.10),
    Dense(16, activation="relu"),
    Dense(1, activation="linear"),
])
model.compile(optimizer="adam", loss="mse", metrics=["mae"])
model.summary()
```

Model: "sequential"

Layer (type)	Output Shape	Param #
gru_2 (GRU)	(None, 32)	3,360
dropout_1 (Dropout)	(None, 32)	0
dense_2 (Dense)	(None, 16)	528
dense_3 (Dense)	(None, 1)	17

Total params: 3,905 (15.25 KB)
Trainable params: 3,905 (15.25 KB)
Non-trainable params: 0 (0.00 B)

```
early_stop = EarlyStopping(
    monitor="val_loss",
    patience=8,
    restore_best_weights=True,
    mode="min",
)

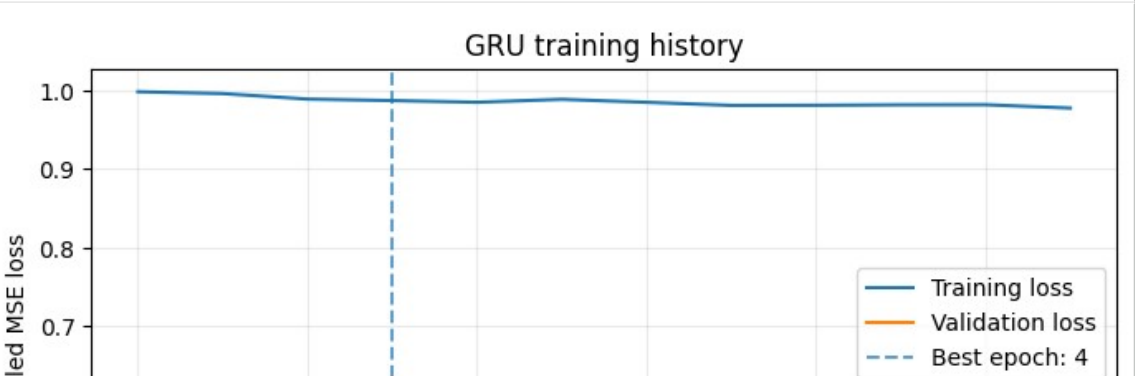
history = model.fit(
    X_train, y_train,
    validation_data=(X_val, y_val),
    epochs=EPOCHS,
```

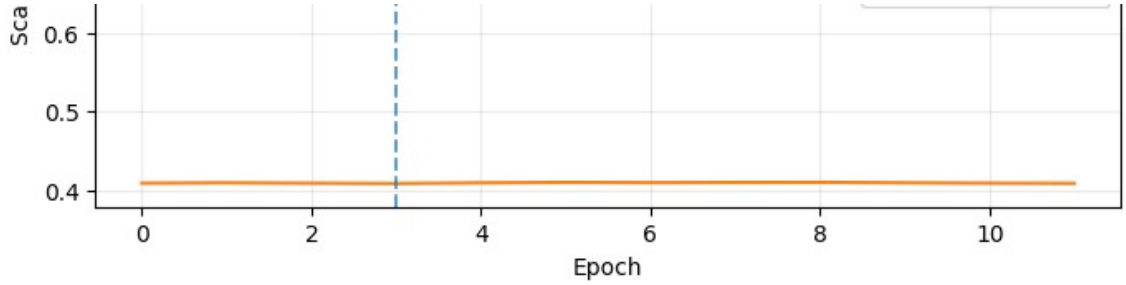
```
    batch_size=BATCH_SIZE,
    callbacks=[early_stop],
    shuffle=False,
    verbose=1,
)

best_epoch = int(np.argmin(history.history["val_loss"]) + 1)
print("Best epoch according to validation loss:", best_epoch)
```

```
Epoch 1/60
102/102 ━━━━━━━━━━━━━━━━━━━━ 2s 9ms/step - loss: 0.9977 - mae: 0.6807 - va
Epoch 2/60
102/102 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.9954 - mae: 0.6799 - va
Epoch 3/60
102/102 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.9884 - mae: 0.6793 - va
Epoch 4/60
102/102 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.9865 - mae: 0.6788 - va
Epoch 5/60
102/102 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.9843 - mae: 0.6780 - va
Epoch 6/60
102/102 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.9881 - mae: 0.6785 - va
Epoch 7/60
102/102 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.9843 - mae: 0.6779 - va
Epoch 8/60
102/102 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.9804 - mae: 0.6783 - va
Epoch 9/60
102/102 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.9805 - mae: 0.6774 - va
Epoch 10/60
102/102 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.9811 - mae: 0.6776 - va
Epoch 11/60
102/102 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.9812 - mae: 0.6782 - va
Epoch 12/60
102/102 ━━━━━━━━━━━━━━━━━━━━ 1s 7ms/step - loss: 0.9770 - mae: 0.6765 - va
Best epoch according to validation loss: 4
```

```
fig, ax = plt.subplots(figsize=(8, 4))
ax.plot(history.history["loss"], label="Training loss")
ax.plot(history.history["val_loss"], label="Validation loss")
ax.axvline(best_epoch - 1, linestyle="--", alpha=0.7, label=f"Best epoch:
ax.set_title("GRU training history")
ax.set_xlabel("Epoch")
ax.set_ylabel("Scaled MSE loss")
ax.grid(True, alpha=0.25)
ax.legend()
plt.show()
```





7. Refit the selected model on all pre-test history

The first fit used data through 2020 for training and 2021–2023 for validation/early stopping. We now **freeze the epoch count selected on that validation period**, refit the scaler on all observations before 2024, and train a fresh GRU on all target dates before 2024. The held-out 2024-onward test period is not used to select features, epochs, or scaling parameters.

```
# Use validation-selected epoch count; refit on all pre-test observations
final_train_mask = sample_dates < TEST_START

final_scaler = StandardScaler()
pretest_return_values = returns.loc[returns.index < TEST_START].to_numpy(
final_scaler.fit(pretest_return_values)

X_final_scaled = final_scaler.transform(X_raw.reshape(-1, 1)).reshape(X_r
y_final_scaled = final_scaler.transform(y_raw).astype(np.float32)
X_final_train = X_final_scaled[final_train_mask]
y_final_train = y_final_scaled[final_train_mask]

# Reinitialize the network so the final evaluation model is trained from
tf.keras.backend.clear_session()
tf.random.set_seed(SEED)
model = Sequential([
    Input(shape=(LOOKBACK, 1)),
    GRU(32),
    Dropout(0.10),
    Dense(16, activation="relu"),
    Dense(1, activation="linear"),
])
model.compile(optimizer="adam", loss="mse", metrics=["mae"])

final_history = model.fit(
    X_final_train,
    y_final_train,
    epochs=best_epoch,
    batch_size=BATCH_SIZE,
    shuffle=False,
```

```
    verbose=1,
)

# Downstream evaluation cells use this scaler/model and untouched test sa
scaler = final_scaler
X_test = X_final_scaled[test_mask]
y_test = y_final_scaled[test_mask]
y_test_actual_pp = y_raw[test_mask].ravel()

print("Final model trained on targets before:", TEST_START.date())
print("Final training samples:", len(X_final_train))
print("Epoch count frozen from validation:", best_epoch)
print("Held-out test samples:", len(X_test))
```

```
Epoch 1/4
125/125 ━━━━━━━━━━━━━━━━━━━━ 2s 6ms/step - loss: 1.0015 - mae: 0.6779
Epoch 2/4
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 0.9939 - mae: 0.6764
Epoch 3/4
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 0.9857 - mae: 0.6749
Epoch 4/4
125/125 ━━━━━━━━━━━━━━━━━━━━ 1s 6ms/step - loss: 0.9839 - mae: 0.6744
Final model trained on targets before: 2024-01-01
Final training samples: 3987
Epoch count frozen from validation: 4
Held-out test samples: 697
```

8. Generate forecasts on the held-out test period

The test period begins on 2024-01-01. Model settings are not tuned using these results. The comparison includes a zero-return baseline, a historical-mean baseline estimated before the test period, and a persistence baseline that predicts the previous observed return.

```
pred_test_scaled = model.predict(X_test, verbose=0).reshape(-1, 1)
pred_gru_pp = scaler.inverse_transform(pred_test_scaled).ravel()

# Baselines are computed only from information available before the test
pretest_returns = returns.loc[returns.index < TEST_START]
historical_mean_pp = float(pretest_returns.mean())
pred_zero_pp = np.zeros_like(y_test_actual_pp)
pred_mean_pp = np.full_like(y_test_actual_pp, historical_mean_pp, dtype=f
# X_raw contains the previous LOOKBACK returns; its final element is the
pred_last_return_pp = X_raw[test_mask, -1, 0].astype(float)

print("Historical mean return used for baseline (pp):", round(historical_
print("Test observations:", len(y_test_actual_pp))
```

```
Historical mean return used for baseline (pp): 0.04857
Test observations: 697
```

9. Evaluate return and price-level errors

MAE and RMSE are reported in percentage points for daily returns. For reference, we also convert each one-step return forecast into a next-close estimate using the previous observed close. These are **one-step-ahead** forecasts, not a recursively simulated multi-day price path.

```
test_dates = sample_dates[test_mask]
previous_close = close.loc[return_dates[LOOKBACK - 1:-1]].to_numpy(dtype=
actual_close = close.loc[test_dates].to_numpy(dtype=float)

predictions = {
    "GRU": pred_gru_pp,
    "Zero-return baseline": pred_zero_pp,
    "Historical-mean baseline": pred_mean_pp,
    "Last-return persistence": pred_last_return_pp,
}

def calculate_metrics(actual_return, predicted_return, actual_price, prev
    predicted_price = previous_price * np.exp(predicted_return / 100.0)
    return {
        "Return MAE (pp)": mean_absolute_error(actual_return, predicted_r
        "Return RMSE (pp)": mean_squared_error(actual_return, predicted_r
        "Close MAE (points)": mean_absolute_error(actual_price, predicted
        "Close RMSE (points)": mean_squared_error(actual_price, predicted
        "Direction accuracy (%)": 100.0 * np.mean(np.sign(actual_return)
    }

metric_rows = []
for model_name, predicted_return in predictions.items():
    row = {"Model": model_name}
    row.update(calculate_metrics(y_test_actual_pp, predicted_return, actu
    if model_name == "Zero-return baseline":
        row["Direction accuracy (%)"] = np.nan  # zero prediction has no
    metric_rows.append(row)

metrics_df = pd.DataFrame(metric_rows).sort_values("Return RMSE (pp)").re
display(metrics_df.round(4))
```

	Model	Return MAE (pp)	Return RMSE (pp)	Close MAE (points)	Close RMSE (points)	Direction accuracy (%)
0	Zero-return baseline	0.7102	1.0247	376.8345	537.8367	NaN
1	Historical-mean baseline	0.7098	1.0251	376.7385	538.1801	51.9369

```
# Model-relative differences vs zero-return baseline; negative means lowe
zero_rmse = float(metrics_df.loc[metrics_df["Model"] == "Zero-return base
```

```
metrics_df["Delta vs zero-return RMSE (pp)"] = metrics_df["Return RMSE (p
display(metrics_df.round(4))
```

	Model	Return MAE (pp)	Return RMSE (pp)	Close MAE (points)	Close RMSE (points)	Direction accuracy (%)	Delta vs zero-return RMSE (pp)
0	Zero-return baseline	0.7102	1.0247	376.8345	537.8367	NaN	0.0000
1	Historical-mean baseline	0.7098	1.0251	376.7385	538.1801	51.9369	0.0003

10. Evaluate consistency across calendar years

The full test summary can hide regime changes. This table reports the same forecast metrics by calendar year so we can see whether model behavior is stable over time.

```
test_prediction_df = pd.DataFrame({
    "Date": test_dates,
    "Actual return (pp)": y_test_actual_pp,
    "Previous close": previous_close,
    "Actual close": actual_close,
})
for name, values in predictions.items():
    test_prediction_df[f"{name} return prediction (pp)"] = values
    test_prediction_df[f"{name} close prediction"] = previous_close * np.

test_prediction_df["Year"] = pd.DatetimeIndex(test_prediction_df["Date"])

by_year_rows = []
for year, group in test_prediction_df.groupby("Year"):
    actual_ret = group["Actual return (pp)"].to_numpy()
    for name in predictions:
        pred_ret = group[f"{name} return prediction (pp)"].to_numpy()
        pred_price = group[f"{name} close prediction"].to_numpy()
        row = {"Year": int(year), "Model": name}
        row.update(calculate_metrics(
            actual_ret,
            pred_ret,
            group["Actual close"].to_numpy(),
            group["Previous close"].to_numpy(),
        ))
        if name == "Zero-return baseline":
            row["Direction accuracy (%)"] = np.nan
        by_year_rows.append(row)

by_year_df = pd.DataFrame(by_year_rows)
```

```
display(by_year_df.round(4))
```

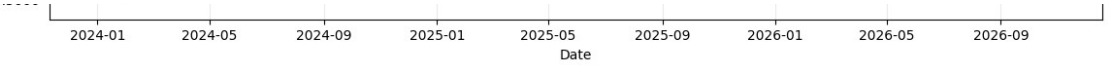
	Year	Model	Return MAE (pp)	Return RMSE (pp)	Close MAE (points)	Close RMSE (points)	Direction accuracy (%)
0	2024	GRU	0.7282	1.1209	362.2029	553.9642	50.8130
1	2024	Zero-return baseline	0.7311	1.1061	363.4806	546.6228	NaN
2	2024	Historical-mean baseline	0.7304	1.1063	363.2009	546.8331	52.0325
3	2024	Last-return persistence	1.0737	1.6915	532.3659	826.1559	53.6585
4	2025	GRU	0.5853	0.7842	313.9620	415.1626	54.4355
5	2025	Zero-return baseline	0.5903	0.7904	316.5445	418.3345	NaN
6	2025	Historical-mean baseline	0.5875	0.7880	315.1131	417.0212	54.4355

11. Visualize one-step-ahead close forecasts

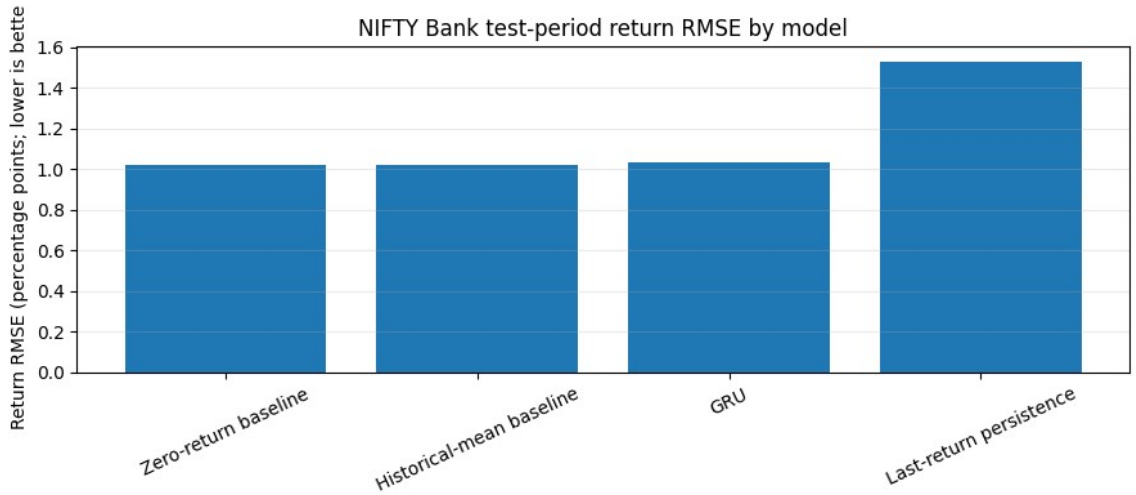
The plot uses a one-step forecast for each test date. It does not mean the model knows the future close path in advance; each forecast uses the prior observed close and prior 20 returns.

```
fig, ax = plt.subplots(figsize=(14, 5))
ax.plot(test_prediction_df["Date"], test_prediction_df["Actual close"], l
ax.plot(test_prediction_df["Date"], test_prediction_df["GRU close predict
ax.set_title("NIFTY Bank: actual vs GRU one-step-ahead close estimate")
ax.set_xlabel("Date")
ax.set_ylabel("Index points")
ax.grid(True, alpha=0.25)
ax.legend()
plt.show()
```






```
fig, ax = plt.subplots(figsize=(9, 4))
plot_df = metrics_df.sort_values("Return RMSE (pp)")
ax.bar(plot_df["Model"], plot_df["Return RMSE (pp)"])
ax.set_title("NIFTY Bank test-period return RMSE by model")
ax.set_ylabel("Return RMSE (percentage points; lower is better)")
ax.tick_params(axis="x", rotation=25)
ax.grid(axis="y", alpha=0.25)
plt.tight_layout()
plt.show()
```



12. Save reproducible outputs

The notebook saves result tables and test predictions under `outputs_experiment_4/`. These files can be downloaded from the Colab Files panel and added to the project's GitHub repository (or linked as release artifacts).

```
OUTPUT_DIR = "outputs_experiment_4"
os.makedirs(OUTPUT_DIR, exist_ok=True)

metrics_df.to_csv(os.path.join(OUTPUT_DIR, "exp4_test_metrics.csv"), inde
by_year_df.to_csv(os.path.join(OUTPUT_DIR, "exp4_yearly_metrics.csv"), in
test_prediction_df.to_csv(os.path.join(OUTPUT_DIR, "exp4_test_predictions

config = {
    "experiment": "Experiment 4 - External-market replication",
    "ticker": TICKER,
    "lookback_sessions": LOOKBACK,
    "features": ["daily log return in percentage points"],
    "train_targets_through": str(TRAIN_END.date()),
    "validation_targets": [str(VAL_START.date()), str(VAL_END.date())],
    "test_targets_from": str(TEST_START.date()),
    "best_epoch": best_epoch,
    "seed": SEED,
    "note": "Method replication on a second index; model weights trained
}
with open(os.path.join(OUTPUT_DIR, "exp4_config.json"), "w", encoding="ut
    json.dump(config, f, indent=2)

print("Saved files:")
for filename in sorted(os.listdir(OUTPUT_DIR)):
    print("-", os.path.join(OUTPUT_DIR, filename))
```

```
Saved files:
- outputs_experiment_4/exp4_config.json
- outputs_experiment_4/exp4_test_metrics.csv
- outputs_experiment_4/exp4_test_predictions.csv
- outputs_experiment_4/exp4_yearly_metrics.csv
```

Experiment 5 — Streamlit Dashboard & Portfolio Packaging

Project: Financial Time-Series Forecasting with GRU/LSTM

This notebook packages the results from Experiments 2–4 into an interactive Streamlit dashboard suitable for a project demo and GitHub portfolio.

Important workflow

- **Recommended:** Copy these cells to the end of your existing master notebook, after Experiments 2, 3 and 4 have run successfully. This allows the notebook to access their saved CSV results.
- Alternatively, run this notebook in a runtime where the folders `outputs/` and `outputs_experiment_4/` already contain the experiment CSVs.
- This dashboard reports historical evaluation. It does **not** make trading

recommendations or claim that the GRU is profitable.

The notebook will create `financial_forecasting_dashboard/` with `app.py`, `requirements.txt`, a README, and the experiment-result CSVs, then package the folder as a ZIP for download and deployment.

1. Imports and dashboard folders

```
from pathlib import Path
import shutil
import textwrap
import zipfile
import pandas as pd

ROOT = Path.cwd()
DASHBOARD_DIR = ROOT / "financial_forecasting_dashboard"
DATA_DIR = DASHBOARD_DIR / "data"
DATA_DIR.mkdir(parents=True, exist_ok=True)

print("Notebook working directory:", ROOT)
print("Dashboard package directory:", DASHBOARD_DIR)
```

```
Notebook working directory: /content
Dashboard package directory: /content/financial_forecasting_dashboard
```

2. Collect the experiment outputs

```
# These files are written by the previous experiment notebooks.
# Experiment 2 and Experiment 3 write into ./outputs/.
# Experiment 4 writes into ./outputs_experiment_4/.
source_files = {
    "exp2_walk_forward_summary.csv": ROOT / "outputs" / "exp2_walk_forwar
    "exp2_walk_forward_fold_metrics.csv": ROOT / "outputs" / "exp2_walk_f
    "exp2_lookback_validation.csv": ROOT / "outputs" / "exp2_lookback_val
    "exp3_feature_validation_metrics.csv": ROOT / "outputs" / "exp3_featu
    "exp4_test_metrics.csv": ROOT / "outputs_experiment_4" / "exp4_test_m
    "exp4_yearly_metrics.csv": ROOT / "outputs_experiment_4" / "exp4_year
    "exp4_test_predictions.csv": ROOT / "outputs_experiment_4" / "exp4_te
}

required_files = {
    "exp2_walk_forward_summary.csv",
    "exp3_feature_validation_metrics.csv",
    "exp4_test_metrics.csv",
    "exp4_yearly_metrics.csv",
}
missing_required = [name for name in required_files if not source_files[n
if missing_required:
    details = "\n".join(f"- {name}: {source_files[name]}" for name in mis
```

```
    details = "\n".join(f"  - {name}: {source_files[name]}" for name in mi
    raise FileNotFoundError(
        "Some required experiment output files are missing:\n"
        + details
        + "\nRun the final output-saving cells for Experiments 2, 3 and 4
          "then re-run this cell."
    )

copied = []
missing_optional = []
for filename, src in source_files.items():
    if src.exists():
        shutil.copy2(src, DATA_DIR / filename)
        copied.append(filename)
    elif filename not in required_files:
        missing_optional.append(filename)

print("Copied result files:")
for filename in copied:
    print(" -", filename)
if missing_optional:
    print("\nOptional files not found (their dashboard sections will be l
    for filename in missing_optional:
        print(" -", filename)
```

```
Copied result files:
 - exp2_walk_forward_summary.csv
 - exp2_walk_forward_fold_metrics.csv
 - exp2_lookback_validation.csv
 - exp3_feature_validation_metrics.csv
 - exp4_test_metrics.csv
 - exp4_yearly_metrics.csv
 - exp4_test_predictions.csv
```

3. Check the packaged result tables

```
for filename in sorted(DATA_DIR.glob("*.csv")):
    df = pd.read_csv(filename)
    print(f"\n{filename.name}: {df.shape[0]} rows × {df.shape[1]} columns
    display(df.head(3))
```

exp2_lookback_validation.csv: 3 rows × 10 columns

	Lookback (sessions)	Best epoch	Validation samples	GRU Return MAE (pp)	GRU Return RMSE (pp)	GRU Close RMSE (points)	GRU Direction accuracy (%)
0	20	5	692	0.658256	0.879143	150.434015	53.757225
1	60	2	692	0.656698	0.879600	150.594890	55.491329
2	30	3	692	0.661369	0.881001	150.806179	54.768786

exp2_walk_forward_fold_metrics.csv: 12 rows × 9 columns

	Fold	Test start	Test end	Model	Return MAE (pp)	Return RMSE (pp)	Close MAE (points)	
0	1	2024-01-09	2024-12-17	GRU	0.606306	0.905421	141.885388	2
1	1	2024-01-09	2024-12-17	Zero-return baseline	0.606143	0.901049	141.674479	2
2	1	2024-01-09	2024-12-17	Last-return persistence	0.924267	1.389351	215.322046	3

exp2_walk_forward_summary.csv: 4 rows × 6 columns

	Model	Return MAE (pp)	Return RMSE (pp)	Close MAE (points)	Close RMSE (points)	Direction accuracy (%)
0	Zero-return baseline	0.611729	0.854056	146.012515	201.984170	NaN
1	Historical-mean baseline	0.611807	0.854369	146.065560	202.106828	51.156069
2	GRU	0.613069	0.856366	146.458355	202.691429	52.745665

exp3_feature_validation_metrics.csv: 3 rows × 12 columns

	Feature set	Feature count	Best epoch	Validation samples	GRU Return MAE (pp)	GRU Return RMSE (pp)	GRU Close RMSE (points)	Di a
0	Returns only	1	5	692	0.657565	0.879027	150.385018	5(
1	Returns + momentum	3	2	692	0.658609	0.880521	150.619809	5⁴
2	Returns + momentum + volatility	5	1	692	0.660029	0.882112	150.987936	5⁴

exp4_test_metrics.csv: 4 rows × 7 columns

	Model	Return MAE (pp)	Return RMSE (pp)	Close MAE (points)	Close RMSE (points)	Direction accuracy (%)	Delta vs zero-return RMSE (pp)
0	Zero-return baseline	0.710185	1.024749	376.834480	537.836702	NaN	0.000000
1	Historical-mean baseline	0.709803	1.025067	376.738511	538.180130	51.936872	0.000318

2	GRU	0.711059	1.032838	377.604780	542.307352	51.506456	0.008089

exp4_test_predictions.csv: 697 rows × 13 columns

Date	Actual return (pp)	Previous close	Actual close	GRU return prediction (pp)	GRU clos predictio

Start coding or generate with AI.